

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN

MÔN: THIẾT KẾ HỆ THỐNG SỐ VỚI HDL
GV viên hướng dẫn: Ths. Ngô Hiếu Trường

ĐỀ TÀI:
DESIGN 256 BIT DATA ADVANCED ENCRYPTION
STANDARD ALGORITHM (AES) USING HDL
THIẾT KẾ MỞ RỘNG THUẬT TOÁN MÃ HÓA DỮ LIỆU AES
256 BIT BẰNG HDL

Sinh viên thực hiện:

Đinh Khánh Đăng - 23520224

Đặng Đình Hiếu - 23520469

LỜI CẢM ƠN

Trong suốt quá trình hoàn thành bài báo cáo môn **THIẾT KẾ HỆ THỐNG SỐ VỚI HDL**:

Chúng em xin chân thành gửi lời cảm ơn đến Trường Đại học Công nghệ Thông tin – Đại học Quốc gia TP.HCM đã tạo điều kiện thuận lợi với hệ thống cơ sở vật chất hiện đại và thư viện phong phú. Những nguồn tài liệu quý giá từ thư viện đã hỗ trợ chúng em rất nhiều trong việc tìm kiếm thông tin phục vụ cho bài báo cáo của mình.

Đặc biệt, chúng em xin gửi lời tri ân sâu sắc nhất đến thầy **Ngô Hiếu Trường** – giảng viên bộ môn, người đã tận tâm giảng dạy và hướng dẫn chúng em trong suốt quá trình học tập. Sự chia sẻ kiến thức quý báu và những bài học bổ ích từ thầy đã trang bị cho chúng em những kỹ năng cần thiết, giúp bài báo cáo này trở nên hoàn thiện hơn.

Nhóm chúng em cũng xin gửi lời cảm ơn đến các thành viên trong nhóm đã cùng nhau nỗ lực và hoàn thành bài báo cáo này. Chúng em đã cùng nhau học hỏi, trao đổi và phối hợp nhịp nhàng để hoàn thành bài báo cáo này một cách tốt nhất. Mỗi thành viên trong nhóm đều đã đóng góp những ý tưởng và kiến thức của mình để làm cho bài báo cáo thêm hoàn thiện.

Xin gửi lời cảm ơn đến các bạn đã quan tâm và theo dõi bài báo cáo của nhóm. Nhóm chúng em hy vọng bài báo cáo này sẽ mang lại những thông tin hữu ích cho các bạn.

Chúng em cũng ý thức rằng do còn thiếu kinh nghiệm và một số hạn chế về kiến thức, bài báo cáo này có thể chưa hoàn hảo. Rất mong nhận được những đóng góp ý kiến và nhận xét từ thầy, để chúng em có thể hoàn thiện hơn trong tương lai.

Cuối cùng, chúng em xin kính chúc thầy **Ngô Hiếu Trường** sức khỏe, hạnh phúc và thành công trong sự nghiệp giảng dạy cũng như trong cuộc sống!

Xin chân thành cảm ơn !

Thành phố Hồ Chí Minh, ngày 20 tháng 5 năm 2026

PHÂN CÔNG NHIỆM VỤ

HỌ VÀ TÊN	MSSV	NHIỆM VỤ	MỨC ĐỘ HOÀN THÀNH
Đặng Đình Hiếu	23520469	Viết module, thiết kế pipeline	100%
Đinh Khánh Đăng	23520224	Viết module, mô phỏng và kiểm thử thiết kế	100%

MỤC LỤC

LỜI CẢM ƠN	2
PHÂN CÔNG NHIỆM VỤ	3
CHƯƠNG I: TỔNG QUAN VỀ ĐỀ TÀI.....	7
I. Giới thiệu đề tài.....	7
II. Mục tiêu nghiên cứu	7
III. Giới hạn đề tài	7
CHƯƠNG 2: THIẾT KẾ CHI TIẾT	8
I. Tổng quan về thiết kế (cấu trúc và sơ đồ khối thiết kế).....	8
1.1 Khởi tạo State và Key Expansion.....	8
1.2. Initial Round (Vòng khởi tạo)	9
1.3. Standard Rounds (Từ vòng 1 đến vòng 13)	9
1.4. Final Round (Vòng 14 - Vòng cuối)	9
II. Chi tiết thiết kế của các khối.....	9
2.1. Key Expansion	9
2.2. AddRoundKey.....	12
2.3. SubBytes.....	14
2.4. ShiftRows	15
2.5. MixColumns.....	17
III. Kiến trúc pipeline AES.....	19
3.1. Ưu điểm về hiệu suất.....	19
3.2. Chiến lược quản lý khóa	19
3.3. Thanh ghi Pipeline	20
CHƯƠNG 3: KIỂM TRA THIẾT KẾ VÀ MÔ PHỎNG.....	21
I. Kiểm tra kết quả chạy từng module	21
1.1 Kết quả chạy testbench của module AddRoundKey trên ModelSim.....	21
1.2. Kết quả chạy testbench của module SubBytes trên ModelSim	22
1.3 Kết quả chạy testbench của module ShiftRows trên ModelSim	23
1.4. Kết quả chạy testbench của module MixColumns trên ModelSim.....	24
1.5. Kết quả chạy testbench của module KeyExpansion trên ModelSim	25
1.6. So sánh kết quả testbench module AddRoundKey, SubBytes, ShiftRows, MixColumns với một số test vector của tài liệu FIPS 197.....	29
1.7. So sánh kết quả testbench module Key Expansion với test vector của tài liệu FIPS 197	30
1.8. Kết quả mô phỏng kiến trúc Pipeline	30
II. Đánh giá tổng quát.....	31

2.1. Pre-simulation.	31
2.2 Post-simulation.	35
III. Total coverage code.	42
CHƯƠNG 4: TỔNG KẾT VÀ ĐÁNH GIÁ QUÁ TRÌNH THỰC HIỆN	43
I. Kết quả đạt được của đề tài.....	43
II. Đánh giá hiệu năng và tài nguyên hệ thống.....	43
III. Những hạn chế còn tồn tại.....	43
IV. Định hướng phát triển tương lai.....	43
CHƯƠNG 5: TÀI LIỆU THAM KHẢO	45

DANH MỤC HÌNH ẢNH

Hình 1: Lưu đồ giải thuật tổng quát.....	8
Hình 2: Ví dụ một word sau khi biến đổi RotWord	10
Hình 3: Ví dụ một word sau khi biến đổi SubWord	10
Hình 4: Ví dụ một word sau khi biến đổi Rcon	10
Hình 5: Lưu đồ giải thuật cho khối key expansion.....	11
Hình 6: Giải thuật cho khối AddRoundKey	13
Hình 7: Giải thuật cho khối SubBytes	14
Hình 8: Giải thuật cho khối ShiftRows	16
Hình 9: Giải thuật cho khối MixColumns	17
Hình 10: Sơ đồ cấu trúc AES Pipeline	19
Hình 11: Kết quả mô phỏng ModelSim của testbench AddRoundKey.v.....	22
Hình 12: Kết quả display output ModelSim của testbench AddRoundKey.v	22
Hình 13: Kết quả mô phỏng ModelSim của testbench SubBytes.v	23
Hình 14: Kết quả display output ModelSim của testbench SubBytes.v	23
Hình 15: Kết quả mô phỏng ModelSim của testbench ShiftRows.v	24
Hình 16: Kết quả display output ModelSim của testbench ShiftRows	24
Hình 17: Kết quả mô phỏng ModelSim của testbench MixColumns.v	25
Hình 18: Kết quả display output ModelSim của testbench MixColumns.v	25
Hình 19: Kết quả mô phỏng ModelSim của testbench KeyExpansion.v.....	29
Hình 20: Kết quả display output ModelSim của testbench MixColumns.v	29
Hình 21: Kết quả mô phỏng ModelSim của testbench top.v	30
Hình 22: Kết quả display output ModelSim của testbench top.v	31
Hình 23: Kết quả tính toán lý thuyết của thuật toán (nist.fips.197)	31
Hình 24: Kết quả pass/fail sẽ được lưu vào file sim_cases.csv và sim_cases.json	33
Hình 25: Kết quả kiểm tra file sim_cases.json: 100% case đã được mã hóa chính xác	33
Hình 26: Kết quả kiểm tra file sim_cases.csv.....	34
Hình 27: Kết quả mô phỏng trên vivado	35
Hình 28: Báo cáo Sử dụng Tài nguyên.....	36
Hình 29: Báo cáo sử dụng chân IO.....	36
Hình 30: Báo cáo định thời.....	37
Hình 31: Đường dẫn dữ liệu dài nhất (Critical Path) trong mạch	38
Hình 32: File CSV kết quả (Post-Simulation CSV)	39
Hình 33: Các file sau khi được tạo	40
Hình 34: Các biểu đồ so sánh chi tiết giữa Pre-Simulation và Post-Simulation từ file summary_compare.csv.....	41
Hình 35_a: Run simulation synthesis/implementation post functional.....	42
Hình 36: Tổng coverage code là 73 %	42

CHƯƠNG I: TỔNG QUAN VỀ ĐỀ TÀI

I. Giới thiệu đề tài

Thuật toán Rijndael, sau này được chuẩn hóa thành Tiêu chuẩn Mã hóa Nâng cao (AES), là một trong những thành tựu rực rỡ nhất của mật mã học đối xứng. Kể từ khi thay thế tiêu chuẩn DES cũ kỹ, AES đã trở thành chuẩn mực toàn cầu để mã hóa dữ liệu điện tử. Đặc biệt, phiên bản AES-256 với quy trình xử lý 14 vòng lặp phức tạp trên khối dữ liệu 128-bit cùng khóa bảo mật 256-bit, mang lại một mức độ an toàn tối đa về mặt toán học. Sự kết hợp hoàn hảo giữa các phép toán thay thế và hoán vị giúp AES-256 không chỉ bảo mật tuyệt đối mà còn tối ưu hóa được hiệu suất trên cả phần cứng lẫn phần mềm. Trong phạm vi bài luận này, chúng ta sẽ cùng giải phẫu chi tiết từng vòng lặp của AES-256, phân tích toán học đằng sau các hộp S-box, và tìm hiểu lý do tại sao cấu trúc này vẫn đứng vững trước thử thách của thời gian.

Việc sử dụng ngôn ngữ mô tả phần cứng verilog đạt hiệu quả hơn rất nhiều so với ngôn ngữ lập trình bình thường vì các thao tác được thực hiện một cách song song thay vì tuần tự.

II. Mục tiêu nghiên cứu

Mục tiêu cốt lõi của bài luận là nghiên cứu thiết kế và hiện thực hóa một cryptographic core cho thuật toán AES-256 tuân thủ nghiêm ngặt các đặc tả kỹ thuật trong tiêu chuẩn NIST FIPS 197.

Để hoàn thành mục tiêu tổng quát, quá trình thực hiện đề tài sẽ tập trung giải quyết các hạng mục kỹ thuật sau:

- Phân tích chi tiết kiến trúc toán học và quy trình xử lý 14 round của chuẩn AES-256 thông qua việc mô xẻ các khối chức năng cốt lõi bao gồm Key Expansion, SubBytes, ShiftRows, MixColumns và AddRoundKey.
- Chuyển đổi các lý thuyết mật mã trừu tượng thành thiết kế logic vật lý thông qua việc lập trình bằng ngôn ngữ mô tả phần cứng Verilog.
- Xây dựng testbench và thực hiện mô phỏng functional simulation trên môi trường ModelSim nhằm kiểm chứng tính chính xác của toàn bộ datapath và control logic.
- Nghiên cứu và tích hợp kiến trúc pipeline vào toàn bộ datapath của hệ thống nhằm tối đa hóa thông lượng dữ liệu đầu ra throughput. Quá trình này đòi hỏi việc thiết kế và chèn các thanh ghi phân tầng pipeline register vào giữa các round xử lý để nâng cao tần số hoạt động tối đa thay vì sử dụng kiến trúc iterative truyền thống.

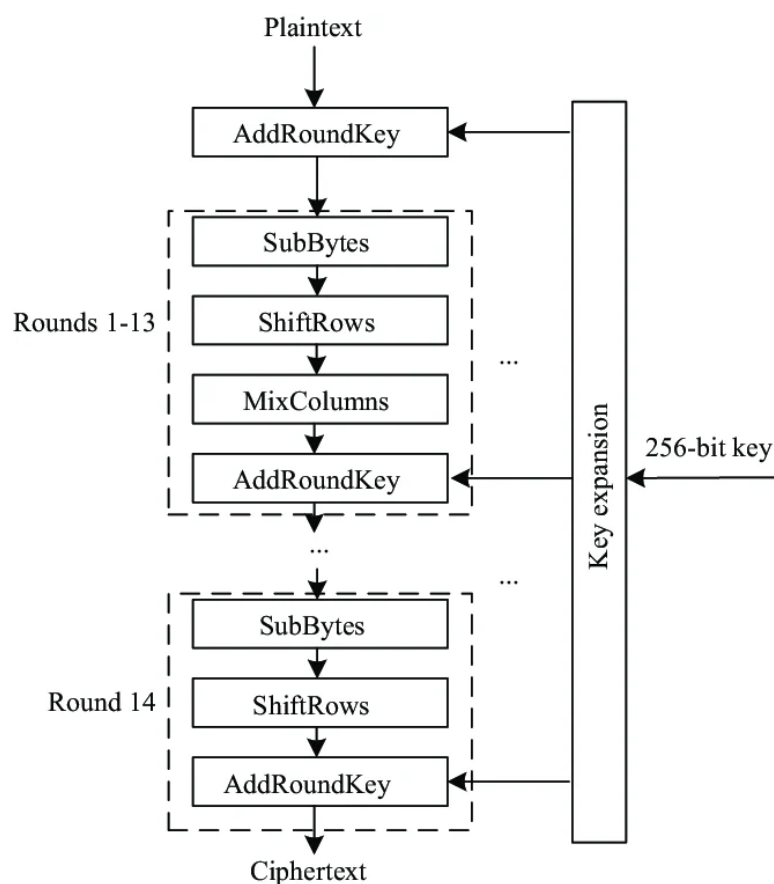
III. Giới hạn đề tài

- Hệ thống mã hóa được thiết kế cố định với cấu hình block size 128 bit và key size 256 bit theo đúng chuẩn FIPS 197 mà không hỗ trợ cấu hình động qua lại giữa các phiên bản AES khác.
- Quá trình kiểm chứng thiết kế dừng lại ở mức mô phỏng timing và logic trên các bộ công cụ chuyên dụng như ModelSim và Quartus thay vì chế tạo vi mạch ASIC thực tế.
- Nền tảng đích của hệ thống được định hướng hoàn toàn cho kiến trúc FPGA nhằm khai thác khả năng xử lý song song.

CHƯƠNG 2: THIẾT KẾ CHI TIẾT

Mục tiêu trọng tâm của chương là trình bày chi tiết quy trình chuyển đổi thuật toán AES-256 thành các khối logic số thông qua ngôn ngữ mô tả phần cứng Verilog. Nội dung sẽ đi sâu vào việc giải phẫu từng thành phần cốt lõi trong hệ thống bao gồm SubBytes, ShiftRows, MixColumns, AddRoundKey và module quản lý khóa Key Expansion.

I. Tổng quan về thiết kế (cấu trúc và sơ đồ khối thiết kế)



Hình 1: Lưu đồ giải thuật tổng quát

Quá trình biến đổi từ plaintext (bản rõ) thành ciphertext (bản mã) trong chuẩn AES được thực hiện thông qua một chuỗi các phép toán lặp đi lặp lại được gọi là các round. Đối với cấu hình AES-256, luồng dữ liệu datapath sẽ phải đi qua tổng cộng 14 round xử lý.

1.1 Khởi tạo State và Key Expansion

- **State Matrix:** Dữ liệu plaintext 128-bit đầu vào được chia nhỏ và tổ chức thành một ma trận 2D kích thước 4x4 (mỗi ô chứa 1 byte). Toàn bộ các phép biến đổi sau đó đều thao tác trực tiếp trên ma trận **State** này.

- **Key Expansion:** Khóa bí mật 256-bit ban đầu không được dùng trực tiếp cho toàn bộ quá trình, mà sẽ đi qua một module mở rộng khóa để tạo ra 15 Round Key riêng biệt, mỗi key có kích thước 128-bit.

1.2. Initial Round (Vòng khởi tạo)

- **AddRoundKey:** Ngay trước khi bước vào các vòng lặp phức tạp, State ban đầu sẽ được thực hiện phép toán logic XOR trực tiếp với Round Key đầu tiên (Key 0). Bước này đóng vai trò "trộn" khóa bảo mật vào dữ liệu ngay từ chu kỳ đầu tiên.

1.3. Standard Rounds (Từ vòng 1 đến vòng 13)

Đây là phần lõi của thuật toán, nơi dữ liệu sẽ tuần tự đi qua 4 khối logic liên tiếp. Trong thiết kế pipeline, 4 khối này thường được đặt nối tiếp nhau trong cùng một tầng:

- **SubBytes:** Phép biến đổi phi tuyến tính. Mỗi byte trong State matrix sẽ được thay thế bằng một byte khác dựa trên một bảng tra cứu cố định gọi là **S-box**. Đây là thành phần quan trọng nhất tạo nên tính an toàn của AES.
- **ShiftRows:** Phép dịch vòng. Các hàng của State matrix sẽ được dịch sang trái với số bước khác nhau (hàng 0 không dịch, hàng 1 dịch 1 byte, hàng 2 dịch 2 byte, hàng 3 dịch 3 byte). Phép toán này giúp phân tán dữ liệu theo chiều ngang.
- **MixColumns:** Phép biến đổi tuyến tính. Mỗi cột của State matrix sẽ được nhân với một đa thức cố định trên trường Galois $GF(2^8)$. Bước này tạo ra sự khuếch tán dữ liệu theo chiều dọc.
- **AddRoundKey:** Tương tự vòng khởi tạo, State hiện tại tiếp tục được XOR với Round Key tương ứng của vòng đó.

1.4. Final Round (Vòng 14 - Vòng cuối)

- Vòng xử lý cuối cùng thực hiện các bước SubBytes, ShiftRows và AddRoundKey, nhưng hoàn toàn loại bỏ khối MixColumns. Việc bỏ qua MixColumns ở vòng cuối cùng được NIST thiết kế có chủ đích nhằm giúp thuật toán mã hóa và giải mã có cấu trúc đối xứng hơn, thuận lợi cho việc thiết kế các module phần cứng dùng chung tài nguyên.

Kết thúc vòng 14, dữ liệu nằm trong State matrix chính là ciphertext hoàn chỉnh và sẵn sàng được đẩy ra ngoài.

II. Chi tiết thiết kế của các khối.

2.1. Key Expansion

2.1.1. Thông số định dạng AES-256

Trong thuật toán AES, đơn vị tính toán cơ bản cho Key Expansion là word (1 word = 32 bit = 4 byte).

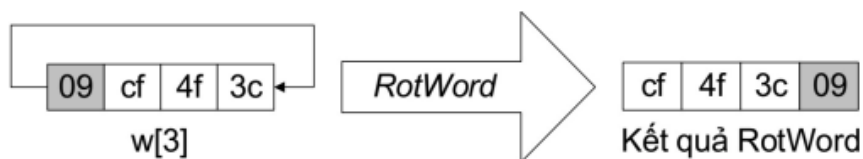
Khóa gốc (Cipher Key): Có kích thước 256 bit, tương đương với 8 word ban đầu, được đánh số từ w_0 đến w_7 .

Khóa vòng (Round Key): Mỗi vòng lặp yêu cầu một khóa 128 bit, tương đương 4 word. Tổng số word cần thiết: Với cấu trúc 14 round tiêu chuẩn cộng thêm 1 Initial Round, hệ thống cần tổng cộng 15 Round Key. Do đó, khối Key Expansion phải tạo ra một mảng gồm 60 word, đánh số từ w_0 đến w_{59} .

2.1.2. Các hàm biến đổi cơ sở

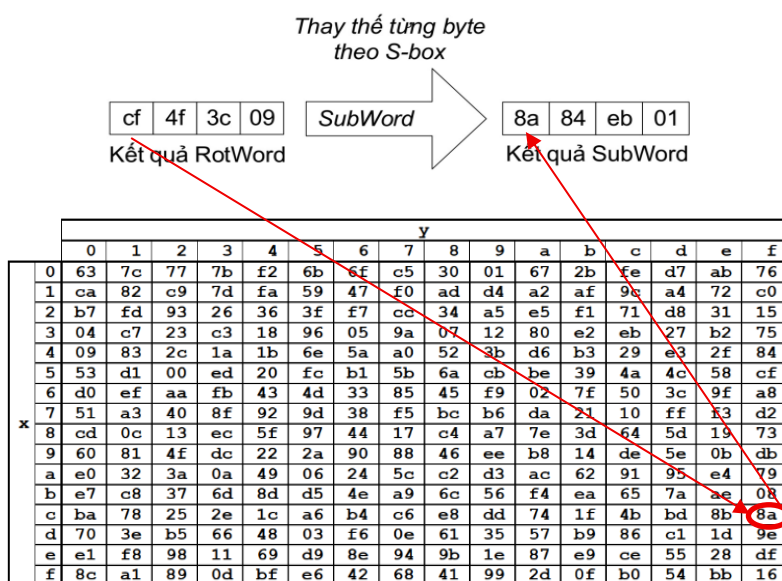
Khối này sử dụng 3 hàm toán học cơ bản để tạo ra tính hỗn loạn cho các khóa sinh ra:

RotWord: Hàm dịch vòng. Hàm này nhận vào một word 4 byte [a, b, c, d] và dịch trái 1 byte để tạo thành [b, c, d, a].



Hình 2: Ví dụ một word sau khi biến đổi RotWord

SubWord: Hàm thay thế phi tuyến tính. Từng byte trong word sẽ được đưa qua bộ S-box để thay thế bằng một byte khác.



Hình 3: Ví dụ một word sau khi biến đổi SubWord

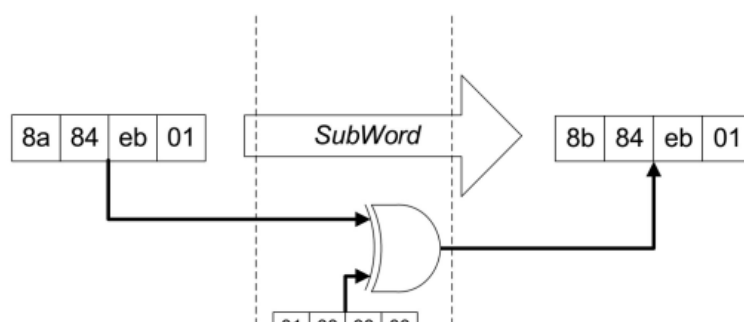
Rcon (Round Constant): Mảng hằng số vòng. Đây là các giá trị được định nghĩa sẵn trên trường Galois $GF(2^8)$, dùng để thực hiện phép XOR với byte đầu tiên của word nhằm triệt tiêu tính đối xứng. Theo định nghĩa của tiêu chuẩn FIPS 197, mỗi phần tử trong mảng Rcon là một word 32 bit. Tuy nhiên, chỉ có byte đầu tiên mang trọng số cao nhất là có giá trị thay đổi, ba byte còn lại luôn được gán cố định bằng 0. Cấu trúc tổng quát sẽ có dạng:

$$Rcon[i] = [RC_i, 0, 0]$$

Quy luật sinh chuỗi giá trị RC_i diễn ra như sau:

Giá trị khởi tạo tại vòng đầu tiên luôn $RC_1 = 0x01$.

Các giá trị tiếp theo được tính bằng cách nhân giá trị trước đó với đa thức x^8 (tương đương với giá trị



Hình 4: Ví dụ một word sau khi biến đổi Rcon

2.1.3. Thuật toán mở rộng khóa cho AES-256

Quá trình sinh ra 60 word từ w_0 đến w_{59} được thực hiện tuần tự. 8 word đầu tiên (w_0 đến w_7) được sao chép trực tiếp từ Cipher Key. Từ w_8 trở đi, hệ thống sinh khóa theo quy tắc sau:

- Để tạo ra word w_i , ta lấy word ngay trước nó là w_{i-1} gán vào một biến tạm temp. Sau đó, temp sẽ bị biến đổi tùy thuộc vào vị trí của chỉ số i .
- Nếu i chia hết cho 8, tức bắt đầu một khối 256 bit mới, hệ thống áp dụng công thức sau:

$$temp = SubWord(RotWord(temp)) \oplus Rcon[i/8]$$

- Nếu i chia hết cho 4, hệ thống áp dụng công thức sau:

$$temp = SubWord(temp)$$

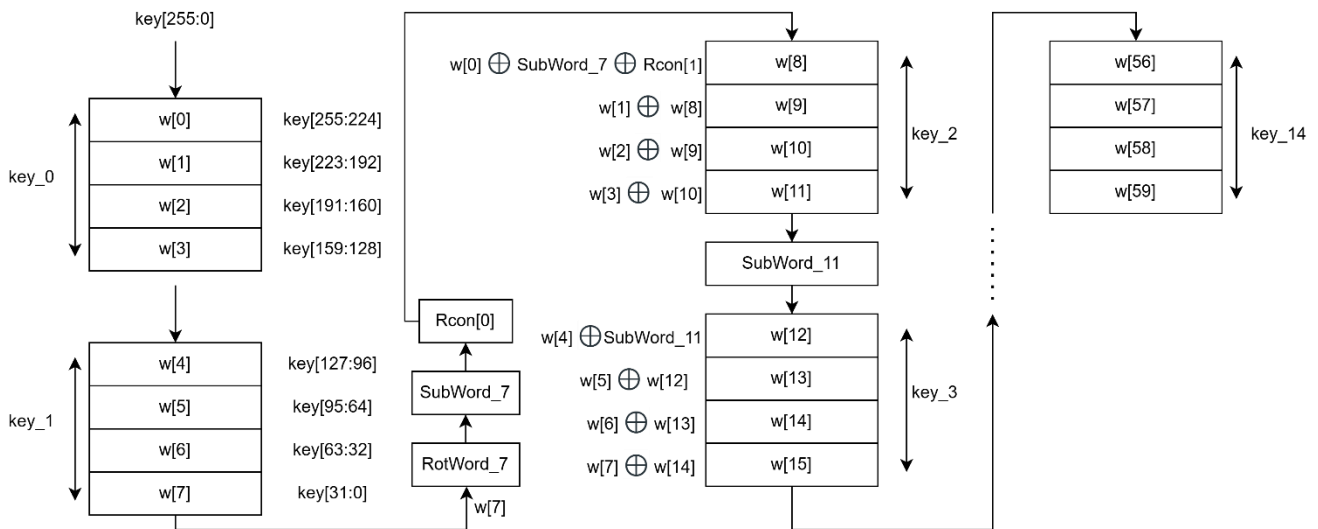
- Với các giá trị i còn lại, temp giữ nguyên:

$$temp = w_{i-1}$$

- Sau khi xử lý xong temp, word mới sẽ được tính như sau:

$$w_i = w_{i-8} \oplus temp$$

Bốn word liên tiếp sinh ra sẽ được ghép lại thành một Round Key 128 bit cấp cho một tầng pipeline tương ứng.



Hình 5: Lưu đồ giải thuật cho khối key expansion

Mã giả cho Key Expansion:

```

CONSTANT Nk = 8;      // Số lượng word của khóa gốc (256 bit = 8 words)
CONSTANT Nb = 4;      // Số lượng word của một khối State (128 bit = 4 words)
CONSTANT Nr = 14;     // Số lượng vòng lặp tiêu chuẩn của AES-256
CONSTANT TOTAL_WORDS = Nb * (Nr + 1);

CONSTANT Rcon[] = {
    0x00000000, 0x01000000, 0x02000000, 0x04000000, 0x08000000,
    0x10000000, 0x20000000, 0x40000000, 0x80000000, 0x1B000000, 0x36000000
};

FUNCTION KeyExpansion(byte CipherKey[32], word W[60]) {
    word temp;
    int i = 0;

    while (i < Nk) {
        W[i] = word(CipherKey[4*i], CipherKey[4*i+1], CipherKey[4*i+2],
CipherKey[4*i+3]);
        i = i + 1;
    }

    i = Nk;

    while (i < TOTAL_WORDS) {
        temp = W[i - 1];

        if (i % Nk == 0) {
            temp = SubWord(RotWord(temp)) ^ Rcon[i / Nk];
        }
        else if (i % Nk == 4) {
            temp = SubWord(temp);
        }

        W[i] = W[i - Nk] ^ temp;

        i = i + 1;
    }
}

```

2.2. AddRoundKey

2.2.1. Tổng quan cách hoạt động

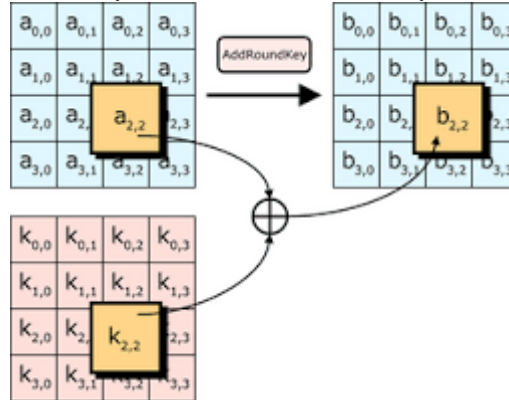
Đây là bước duy nhất trong thuật toán AES thực sự có sự tham gia của Khóa bí mật (Secret Key), đóng vai trò quyết định trong việc bảo mật dữ liệu.

Mục đích: Trộn lẫn dữ liệu hiện tại (gọi là Trạng thái - State) với Khóa của vòng tương ứng (Round Key).

Trong lý thuyết trường hữu hạn $Galois\ GF(2^8)$ được AES sử dụng, phép "cộng" (Add) tương đương với phép toán logic XOR (Exclusive OR) bit-by-bit. Do đó, khối này không sử dụng bộ cộng (adder) thông thường có nhớ (carry), mà chỉ thực hiện tính XOR.

Cấu trúc Input/Output:

- *Input 1 (data / state)*: Khối dữ liệu đầu vào kích thước 128-bit (16 byte). Ở vòng đầu tiên (Round 0), đây chính là **plaintext**. Ở các vòng tiếp theo, nó là kết quả từ bước trước đó (MixColumns hoặc ShiftRows).
- *Input 2 (key / round_key)*: Khóa vòng 128-bit được tạo ra từ bộ mở rộng khóa (Key Expansion).
- *Output (out)*: Khối dữ liệu mới 128-bit thu được sau khi đã XOR.



Hình 6: Giải thuật cho khối AddRoundKey

Tính chất: Khối này có tính chất đối xứng tự nhiên. Nghĩa là quá trình mã hóa (Encryption) và giải mã (Decryption) đều dùng chung một cấu trúc AddRoundKey, vì:

$$(A \text{ xor } B) \text{ xor } B = A.$$

2.2.2 Hiện thực code

Định nghĩa Interface và assign

- 2 cổng input: Dữ liệu (128-bit) và Khóa (128-bit).
- 1 cổng output: Dữ liệu đã trộn khóa (128-bit).

Vì AddRoundKey chỉ là phép toán XOR song song đồng thời trên tất cả 128 bit, mạch phần cứng không cần sử dụng xung clock.

```
module addRoundKey(data, out, key);
```

```
input [127:0] data;
```

```
input [127:0] key;
```

```
output [127:0] out;
```

```
assign out = key ^ data;
```

```
endmodule
```

Tích hợp vào hệ thống:

Trong file kiến trúc tổng (*top.v*), module AddRoundKey được khởi tạo nhiều lần tương ứng với số vòng lặp của thuật toán AES-256 (tổng cộng 15 lần cho vòng 0 đến vòng 14).

Vòng sơ khởi (Round 0): Dữ liệu vào là plaintext, kết hợp trực tiếp với khóa gốc RoundKey[0]:

Top.v

```
addRoundKey ark0(.data(plaintext), .key(RoundKey[0]), .out(AfterAddRoundKey[0]));
```

Các vòng lặp chính (Round 1 - Round 14): Dữ liệu vào nhận từ ngõ ra của khối trước đó (thường là khối MixColumns hoặc ShiftRows) và trộn với Khóa của vòng đó (RoundKey[i]):

```

Top.v
addRoundKey ark1(.data(AfterMixColumns[0]), .key(RoundKey[1]),.out(AfterAddRoundKey[1]));

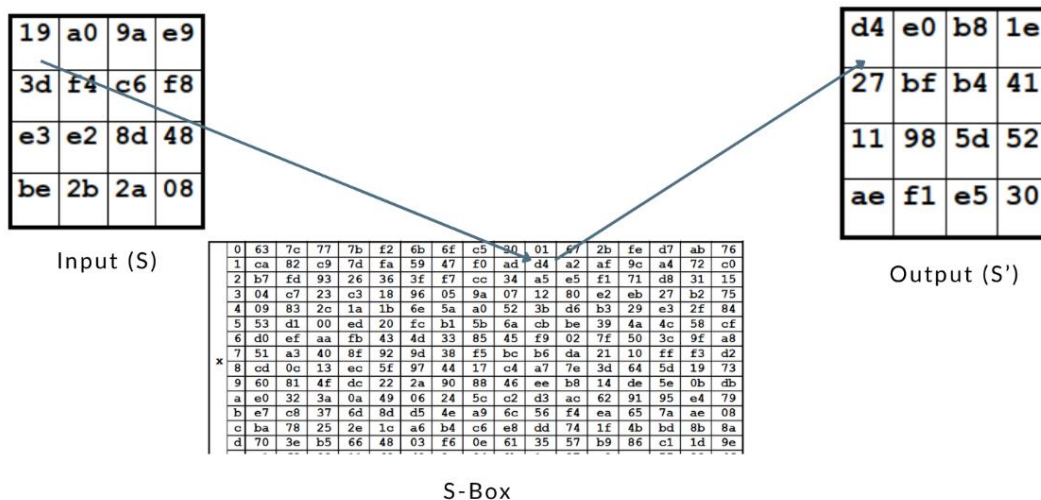
```

Đánh giá tài nguyên: Đoạn code trên khi được phần mềm tổng hợp (Synthesis) có thể sẽ tiêu tốn chính xác 128 cổng logic và delay gần như bằng 0.

2.3. SubBytes

2.3.1. Tổng quan về cách hoạt động

SubBytes là phép biến đổi thay thế độc lập từng byte của ma trận State dựa trên một bảng tra cứu cố định S-box kích thước 16x16. Các giá trị trong bảng S-box được thiết kế chặt chẽ bằng cách kết hợp phép nghịch đảo nhân trên trường hữu hạn $GF(2^8)$ và phép biến đổi Affine. Sự kết hợp này nhằm mục đích triệt tiêu hoàn toàn mọi đặc tính tuyến tính.



Hình 7: Giải thuật cho khối SubBytes

Cấu trúc input/output:

- **Tín hiệu ngõ vào:** state_in có kích thước 128 bit. Tín hiệu này nhận toàn bộ dữ liệu của ma trận State từ khối AddRoundKey hoặc từ ngõ vào ban đầu chuyển tới.
- **Tín hiệu ngõ ra:** state_out có kích thước 128 bit. Tín hiệu này đẩy ma trận State mới sau khi đã thay thế hoàn tất toàn bộ các byte sang cho khối ShiftRows xử lý tiếp.

2.3.2. Hiện thực hóa trên code Verilog:

```

SubBytes.v
module sub_bytes(
    input wire [127:0] state_in,
    output wire [127:0] state_out
);
    sbx s0_0(.index(state_in[127:120]), .out(state_out[127:120]));
    sbx s0_1(.index(state_in[119:112]), .out(state_out[119:112]));
    sbx s0_2(.index(state_in[111:104]), .out(state_out[111:104]));
    sbx s0_3(.index(state_in[103:96]), .out(state_out[103:96]));

```

```
sbox s1_0(.index(state_in[95:88]), .out(state_out[95:88]));
sbox s1_1(.index(state_in[87:80]), .out(state_out[87:80]));
sbox s1_2(.index(state_in[79:72]), .out(state_out[79:72]));
sbox s1_3(.index(state_in[71:64]), .out(state_out[71:64]));
sbox s2_0(.index(state_in[63:56]), .out(state_out[63:56]));
sbox s2_1(.index(state_in[55:48]), .out(state_out[55:48]));
sbox s2_2(.index(state_in[47:40]), .out(state_out[47:40]));
sbox s2_3(.index(state_in[39:32]), .out(state_out[39:32]));
sbox s3_0(.index(state_in[31:24]), .out(state_out[31:24]));
sbox s3_1(.index(state_in[23:16]), .out(state_out[23:16]));
sbox s3_2(.index(state_in[15:8]), .out(state_out[15:8]));
sbox s3_3(.index(state_in[7:0]), .out(state_out[7:0]));
endmodule
```

Tích hợp vào file hệ thống

Trong file kiến trúc tổng (*top.v*), module SubBytes được khởi tạo 14 lần tương ứng với 14 vòng lặp của thuật toán AES-256. Sau khi đã thay thế hoàn tất toàn bộ các byte sẽ chuyển sang cho khối ShiftRows xử lý tiếp.

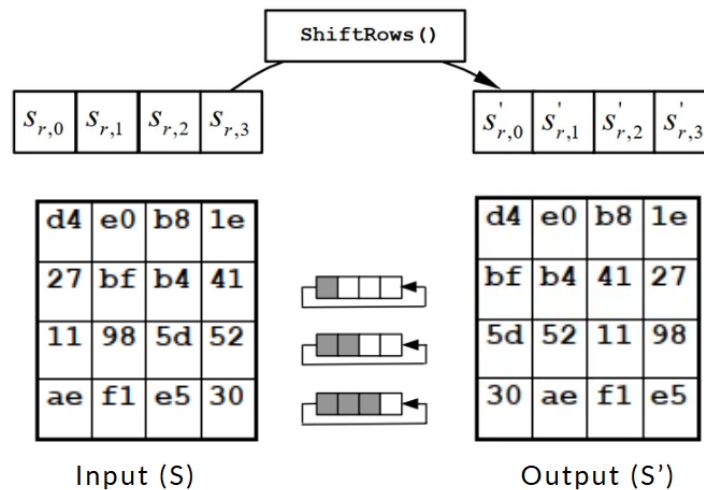
Top.v
sub_bytes sb1(.state_in(state[0]), .state_out(AfterSubBytes[0]));

2.4. ShiftRows

2.4.1. Tổng quan cách hoạt động

Phép ShiftRows thực hiện phép dịch vòng left shift đối với các byte trên từng hàng ngang của ma trận state kích thước 4x4 với biên độ cụ thể như sau:

- **Hàng 0:** Giữ nguyên vị trí ban đầu và không chịu bất kỳ sự dịch chuyển nào.
- **Hàng 1:** Dịch vòng sang trái 1 byte. Byte đầu tiên của hàng sẽ bị đẩy xuống cuối hàng.
- **Hàng 2:** Dịch vòng sang trái 2 byte.
- **Hàng 3:** Dịch vòng sang trái 3 byte.



Hình 8: Giải thuật cho khối ShiftRows

Cấu trúc Input/Output:

- **Input:** vector 128 bit là trạng thái của ma trận 4x4 hiện tại trước khi thực hiện dịch hàng
- **Output:** vector 128 bit là trạng thái sau khi đã hoàn tất phép dịch vòng

2.4.2. Hiện thực hóa trên code Verilog:

ShiftRows.v
<pre> module shift_rows(input wire [127:0] state_in, output wire [127:0] state_out); assign state_out = { state_in[127:120], state_in[87:80], state_in[47:40], state_in[7:0], state_in[95:88], state_in[55:48], state_in[15:8], state_in[103:96], state_in[63:56], state_in[23:16], state_in[111:104], state_in[71:64], state_in[31:24], state_in[119:112], state_in[79:72], state_in[39:32] }; endmodule </pre>

Tích hợp vào hệ thống

Trong file kiến trúc tổng (*top.v*), module ShiftRows được khởi tạo 14 lần tương ứng với 14 vòng lặp của thuật toán AES-256. Sau khi đã thay thế hoàn tất toàn bộ các byte sẽ chuyển sang cho khối MixColumns xử lý tiếp.

Top.v
<pre> shift_rows srl(.state_in(AfterSubBytes[0]), .state_out(AfterShiftRows[0])); </pre>

2.5. MixColumns

2.5.1. Tổng quan cách hoạt động

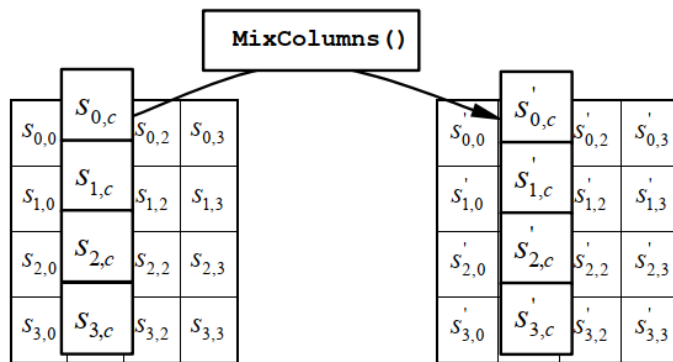
Trong khối MixColumns, mỗi cột của State matrix bao gồm 4 byte được coi là một đa thức bậc 3 với các hệ số thuộc trường Galois $GF(2^8)$. Đa thức này sẽ được nhân với một đa thức cố định $a(x)$ theo modulo $x^4 + 1$. Đa thức cố định này được NIST định nghĩa là:

$$a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}$$

Phép nhân đa thức này có thể được biểu diễn dưới dạng nhân ma trận. Mỗi cột c của State matrix đầu vào sẽ nhân với một ma trận hằng số để tạo ra cột dữ liệu mới ở ngõ ra:

Đặt $s'(x) = a(x) \otimes s(x)$:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}$$



Hình 9: Giải thuật cho khối MixColumns

Cấu trúc Input/Output:

- **Input:** vector 128 bit là trạng thái của ma trận 4x4 hiện tại trước khi thực hiện trộn cột
- **Output:** vector 128 bit là trạng thái sau khi đã hoàn tất phép trộn cột

2.5.2. Hiện thực hóa trên code Verilog:

MixColumns.v
<pre> module MixColumns(input [127:0] state_in, output [127:0] state_out); </pre>

```

function [7:0] mb2;
    input [7:0] x;
    begin
        if(x[7] == 1)
            mb2 = ((x << 1) ^ 8'h1b);
        else
            mb2 = x << 1;
        end
    endfunction

function [7:0] mb3;
    input [7:0] x;
    begin
        mb3 = mb2(x) ^ x;
    end
endfunction

genvar i;
generate
    for(i = 0; i < 4; i = i + 1) begin : m_col
        assign state_out[(i*32 + 24)+:8] = mb2(state_in[(i*32 + 24)+:8]) ^
mb3(state_in[(i*32 + 16)+:8]) ^ state_in[(i*32 + 8)+:8] ^ state_in[i*32+:8];
        assign state_out[(i*32 + 16)+:8] = state_in[(i*32 + 24)+:8] ^ mb2(state_in[(i*32
+ 16)+:8]) ^ mb3(state_in[(i*32 + 8)+:8]) ^ state_in[i*32+:8];
        assign state_out[(i*32 + 8)+:8] = state_in[(i*32 + 24)+:8] ^ state_in[(i*32 +
16)+:8] ^ mb2(state_in[(i*32 + 8)+:8]) ^ mb3(state_in[i*32+:8]);
        assign state_out[i*32+:8] = mb3(state_in[(i*32 + 24)+:8]) ^ state_in[(i*32
+ 16)+:8] ^ state_in[(i*32 + 8)+:8] ^ mb2(state_in[i*32+:8]);
    end
endgenerate

endmodule

```

Tích hợp vào hệ thống

Trong file kiến trúc tổng (*top.v*), module ShiftRows được khởi tạo 14 lần tương ứng với 14 vòng lặp của thuật toán AES-256. Sau khi đã thay thế hoàn tất toàn bộ các byte sẽ chuyển sang cho khối AddRoundKey xử lý tiếp.

Top.v
MixColumns mc1(.state_in(AfterShiftRows[0]), .state_out(AfterMixColumns[0]));

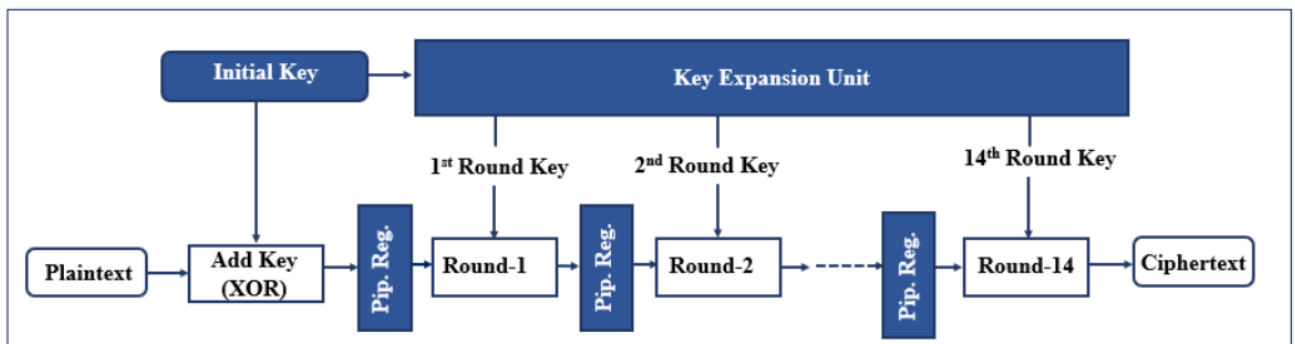
III. Kiến trúc pipeline AES

Cấu trúc pipeline trong thiết kế này dựa trên việc chèn các thanh ghi phân tầng giữa mỗi vòng lặp xử lý. Mỗi tầng trong hệ thống sẽ đảm nhận thực hiện các phép biến đổi cụ thể bao gồm SubBytes, ShiftRows, MixColumns và AddRoundKey. Khi một tầng hoàn tất công việc, dữ liệu sẽ được lưu vào các thanh ghi đồng bộ theo tín hiệu xung nhịp, cho phép tầng đó tiếp nhận khối dữ liệu kế tiếp ngay lập tức.

3.1. Ưu điểm về hiệu suất

Việc triển khai pipeline mang lại khả năng xử lý song song vượt trội:

- **Tăng thông lượng dữ liệu:** Hệ thống có thể xuất ra một khối ciphertext hoàn chỉnh sau mỗi chu kỳ xung nhịp thay vì phải chờ đợi tổng số vòng lặp kết thúc.
- **Tối ưu hóa tần số hoạt động:** Bằng cách chia nhỏ các logic tổ hợp phức tạp thành các phân đoạn ngắn hơn, tần số xung nhịp tối đa của thiết kế trên FPGA được cải thiện đáng kể so với kiến trúc iterative truyền thống.



Hình 10: Sơ đồ cấu trúc AES Pipeline

3.2. Chiến lược quản lý khóa

Thay vì triển khai Key Expansion dưới dạng logic tổ hợp tốn kém (tính toán 13 khóa cùng lúc, Delay sẽ là từ Word[7] đến Word[59]), đề án này sử dụng mô hình tuần tự để giảm độ delay của mạch và tăng *Fmax*:

- **Tính toán tuần tự:** Thuật toán Key Expansion được triển khai trên một khối logic lập, chỉ tính toán 4 Word ($W[i]$) hoặc một Khóa Vòng (K_i) sau mỗi chu kỳ clock.
- **Thời gian sinh Key:** Với 13 Khóa Vòng, sẽ mất khoảng 14 chu kỳ clock để sinh ra toàn bộ các khóa cần thiết từ khóa gốc 256 bit.
- **Module Key_Expansion** sinh tất cả 13 round keys song song với dữ liệu.
- Các round keys được kết nối trong wire array RoundKey[0:14].

- Mỗi stage pipeline truy cập trực tiếp vào round key tương ứng thông qua kết nối wire.
- Roundkey được lấy ra tương ứng cho từng Round, giảm độ phức tạp thiết kế.

3.3. Thanh ghi Pipeline

Các thanh ghi 128 bit được chèn vào giữa các vòng (Round N và Round $N + 1$) để đồng bộ hóa dữ liệu và lưu trữ dữ liệu cho xung clock kế tiếp. Tại mỗi sườn lên của xung nhịp đồng hồ, toàn bộ dữ liệu trung gian sinh ra từ các mạch logic tổ hợp sẽ được chốt đồng loạt vào các thanh ghi này. Nhờ các thanh ghi này, ngay khi Round N hoàn thành, Round $N + 1$ có thể bắt đầu xử lý, trong khi Round N chuẩn bị nhận khối dữ liệu mới.

CHƯƠNG 3: KIỂM TRA THIẾT KẾ VÀ MÔ PHỎNG

Nội dung chương sẽ lần lượt đi qua các kịch bản mô phỏng từ mức độ module đơn lẻ cho đến việc đánh giá toàn diện hoạt động của Top Module trên các phần mềm mô phỏng khác.

1. Kiểm tra kết quả chạy từng module

Các module đã thiết kế sẽ được chạy mô phỏng trên ModelSim và so sánh với các test vector của FIPS 197.

1.1 Kết quả chạy testbench của module AddRoundKey trên ModelSim

1.1.1. File testbench

Trong testbench của AddRoundKey, input của module bao gồm 4 round key 128 bit và 4 khối dữ liệu 128 bit cần được mã hóa. Mỗi khối dữ liệu cùng với một round key lần lượt được input vào module. Output của mỗi khối dữ liệu là 128 bit đã được XOR với round key.

AddRoundKey_tb.v
<pre>module AddRoundKey_tb(); wire [127:0] out; reg [127:0] data, key; addRoundKey ark(.data(data), .key(key), .out(out)); task run_case(input [127:0] data_in, input [127:0] key_in); begin data = data_in; key = key_in; #10; \$display("Data: %h", data); \$display("Key: %h", key); \$display("Output: %h", out); \$display("-----"); end endtask initial begin run_case(128'h00112233445566778899aabbccddeeff, 128'h000102030405060708090a0b0c0d0e0f); run_case(128'h5f72641557f5bc92f7be3b291db9f91a, 128'h101112131415161718191a1b1c1d1e1f);</pre>

```

run_case(128'hbd2a395d2b6ac438d192443e615da195,
         128'ha573c29fa176c498a97fce93a572c09c);

run_case(128'h810dce0cc9db8172b3678c1e88a1b5bd,
         128'h1651a8cd0244bedala5da4c10640bade);
end
endmodule

```

1.1.2. Kết quả

/AddRoundKey_tb/out	975c66c1cb9f3fa...	0010203040506070...	4f63760643e0aa85e...	1859fbc28a1c00a07...	975c66c1cb9f3fa...
/AddRoundKey_tb/data	810dce0cc9db81...	0011223344556677...	5f72641557f5bc92f...	bd2a395d2b6ac438d...	810dce0cc9db81...
/AddRoundKey_tb/key	1651a8cd0244be...	0001020304050607...	1011121314151617...	a573c29fa176c498a...	1651a8cd0244be...

Hình 11: Kết quả mô phỏng ModelSim của testbench AddRoundKey.v

```

# Data: 00112233445566778899aabbccddeeff
# Key: 000102030405060708090a0b0c0d0e0f
# Output: 00102030405060708090a0b0c0d0e0f0
# -----
# Data: 5f72641557f5bc92f7be3b291db9f91a
# Key: 101112131415161718191a1b1c1d1e1f
# Output: 4f63760643e0aa85efa7213201a4e705
# -----
# Data: bd2a395d2b6ac438d192443e615da195
# Key: a573c29fa176c498a97fce93a572c09c
# Output: 1859fbc28a1c00a078ed8aadcd42f6109
# -----
# Data: 810dce0cc9db8172b3678c1e88a1b5bd
# Key: 1651a8cd0244bedala5da4c10640bade
# Output: 975c66c1cb9f3fa8a93a28df8ee10f63
# -----

```

Hình 12: Kết quả display output ModelSim của testbench AddRoundKey.v

1.2. Kết quả chạy testbench của module SubBytes trên ModelSim

Trong testbench của SubBytes, input của module bao gồm 3 khối dữ liệu 128 bit cần được mã hóa. Mỗi khối dữ liệu lần lượt được input vào module. Output của mỗi khối dữ liệu là 128 bit đã được ánh xạ qua bảng S-Box.

1.2.1. File test bench

SubBytes_tb.v
<pre> module sub_bytes_tb; reg [127:0] state_in; wire [127:0] state_out; sub_bytes sb(.state_in(state_in), .state_out(state_out)); task run_case(input [127:0] data_in); begin </pre>

```

state_in = data_in;
#10; // Wait for the sub bytes to complete
$display("Input: %h", state_in);
$display("Output: %h", state_out);
$display("-----");
end
endtask
initial begin
    run_case(128'h00102030405060708090a0b0c0d0e0f0);

    run_case(128'h4f63760643e0aa85efa7213201a4e705);

    run_case(128'h1859fbc28a1c00a078ed8aad42f6109);
end
endmodule

```

1.2.2. Kết quả

/sub_bytes_tb/state_in	1859fbc28a1c00...	00102030405060708090a0b...	4f63760643e0aa85efa7213...	1859fbc28a1c00a078ed8aa...
/sub_bytes_tb/state_out	adcb0f257e9c63...	63cab7040953d051cd60e0e...	84fb386f1ae1ac97df5cfd23...	adcb0f257e9c63e0bc557e9...

Hình 13: Kết quả mô phỏng ModelSim của testbench SubBytes.v

```

# Input: 00102030405060708090a0b0c0d0e0f0
# Output: 63cab7040953d051cd60e0e7ba70e18c
# -----
# Input: 4f63760643e0aa85efa7213201a4e705
# Output: 84fb386f1ae1ac97df5cfd237c49946b
# -----
# Input: 1859fbc28a1c00a078ed8aad42f6109
# Output: adcb0f257e9c63e0bc557e951c15ef01
# -----

```

Hình 14: Kết quả display output ModelSim của testbench SubBytes.v

1.3 Kết quả chạy testbench của module ShiftRows trên ModelSim

Trong testbench của ShiftRows, input của module bao gồm 3 khối dữ liệu 128 bit cần được mã hóa. Mỗi khối dữ liệu lần lượt được input vào module. Output của mỗi khối dữ liệu là 128 bit đã được dịch hàng (hàng 0 không dịch, hàng 1 dịch 1 byte, hàng 2 dịch 2 byte, hàng 3 dịch 3 byte).

1.3.1 File mô phỏng

ShiftRows tb.v
<pre> module shift_rows_tb(); wire [127:0] state_out; reg [127:0] state_in; shift_rows sr(.state_in(state_in), .state_out(state_out)); </pre>

```

task run_case(input [127:0] data_in);
begin
    state_in = data_in;
    #10; // Wait for the shift rows to complete
    $display("State In: %h", state_in);
    $display("State Out: %h", state_out);
    $display("-----");
end
endtask
initial begin
    run_case(128'h63cab7040953d051cd60e0e7ba70e18c);
    run_case(128'h84fb386f1ae1ac97df5cfd237c49946b);
    run_case(128'hadcb0f257e9c63e0bc557e951c15ef01);
end
endmodule

```

1.3.2. Kết quả

/shift_rows_tb/state_out	6353e08c0960e1...	6353e08c0960e104cd70b75...	84e1fd6b1a5c946fdf493897...	ad9c7e017e55ef25bc150...
/shift_rows_tb/state_in	63cab7040953d0...	63cab7040953d051cd60e0e...	84fb386f1ae1ac97df5cfd23...	adcb0f257e9c63e0bc557...

Hình 15: Kết quả mô phỏng ModelSim của testbench ShiftRows.v

```

# State In: 63cab7040953d051cd60e0e7ba70e18c
# State Out: 6353e08c0960e104cd70b751bacad0e7
# -----
# State In: 84fb386f1ae1ac97df5cfd237c49946b
# State Out: 84e1fd6b1a5c946fdf4938977cfbac23
# -----
# State In: adcb0f257e9c63e0bc557e951c15ef01
# State Out: ad9c7e017e55ef25bc150fe01ccb6395
# -----

```

Hình 16: Kết quả display output ModelSim của testbench ShiftRows

1.4. Kết quả chạy testbench của module MixColumns trên ModelSim

Trong testbench của MixColumns, input của module bao gồm 3 khối dữ liệu 128 bit cần được mã hóa. Mỗi khối dữ liệu lần lượt được input vào module. Output của mỗi khối dữ liệu là 128 bit đã được nhân với ma trận hằng số.

1.4.1 File mô phỏng

MixColumns_tb.v
<pre> module MixColumns_tb(); wire [127:0] out; reg [127:0] data; MixColumns mc(.state_in(data), .state_out(out)); task run_case(input [127:0] data_in); </pre>


```

begin
    data = data_in;
    #10;
    $display("Data: %h", data);
    $display("Output: %h", out);
    $display("-----");
end
endtask
initial begin
    run_case(128'h6353e08c0960e104cd70b751bacad0e7);
    run_case(128'h84e1fd6b1a5c946fdf4938977cfbac23);
    run_case(128'had9c7e017e55ef25bc150fe01ccb6395);
end
endmodule

```

1.4.2 Kết quả

		/MixColumns_tb/out	810dce0cc9db81...	5f72641557f5bc92f7be3b29...	bd2a395d2b6ac438d192443e615da195	810dce0cc9...
		/MixColumns_tb/data	ad9c7e017e55ef...	6353e08c0960e104cd70b75...	84e1fd6b1a5c946fdf4938977cfbac23	ad9c7e017e...

Hình 17: Kết quả mô phỏng ModelSim của testbench MixColumns.v

```

# Data: 6353e08c0960e104cd70b751bacad0e7
# Output: 5f72641557f5bc92f7be3b29ldb9f91a
# -----
# Data: 84e1fd6b1a5c946fdf4938977cfbac23
# Output: bd2a395d2b6ac438d192443e615da195
# -----
# Data: ad9c7e017e55ef25bc150fe01ccb6395
# Output: 810dce0cc9db8172b3678cle88alb5bd
# -----

```

Hình 18: Kết quả display output ModelSim của testbench MixColumns.v

1.5. Kết quả chạy testbench của module KeyExpansion trên ModelSim

1.5.1. File testbench

Trong testbench của KeyExpansion, input của module gồm 1 khóa ban đầu là 256 bit. Output là 15 khóa, từ key_0 đến key_14. Key_0 và key_1 được tạo ra trong chu kỳ xung clock đầu tiên, mỗi khóa từ key_2 đến key_14 lần lượt được tạo ra tại mỗi cạnh lên của những xung clock sau đó.

KeyExpansion_tb.v
<pre> module key_expansion_tb(); wire [127:0] key_0,key_1,key_2,key_3,key_4,key_5,key_6,key_7,key_8,key_9,key_10,key_11,key_ 12,key_13,key_14; reg [255:0] key; reg clk; key_expansion ke(.key(key), </pre>

```

        .key_0(key_0),
        .key_1(key_1),
        .key_2(key_2),
        .key_3(key_3),
        .key_4(key_4),
        .key_5(key_5),
        .key_6(key_6),
        .key_7(key_7),
        .key_8(key_8),
        .key_9(key_9),
        .key_10(key_10),
        .key_11(key_11),
        .key_12(key_12),
        .key_13(key_13),
        .key_14(key_14),
        .clk(clk)
    );
initial begin
    clk = 0;
    forever begin
        #5 clk = ~clk;
    end
end

initial begin
    key =
256'h000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f;
    #160; // Wait for the key expansion to complete
    $display("Initial Key: %h", key);
    $display("Key 0: %h", key_0);
    $display("Key 1: %h", key_1);
    $display("Key 2: %h", key_2);
    $display("Key 3: %h", key_3);
    $display("Key 4: %h", key_4);
    $display("Key 5: %h", key_5);
    $display("Key 6: %h", key_6);
    $display("Key 7: %h", key_7);
    $display("Key 8: %h", key_8);
    $display("Key 9: %h", key_9);
    $display("Key 10: %h", key_10);
    $display("Key 11: %h", key_11);
    $display("Key 12: %h", key_12);
    $display("Key 13: %h", key_13);
    $display("Key 14: %h", key_14);

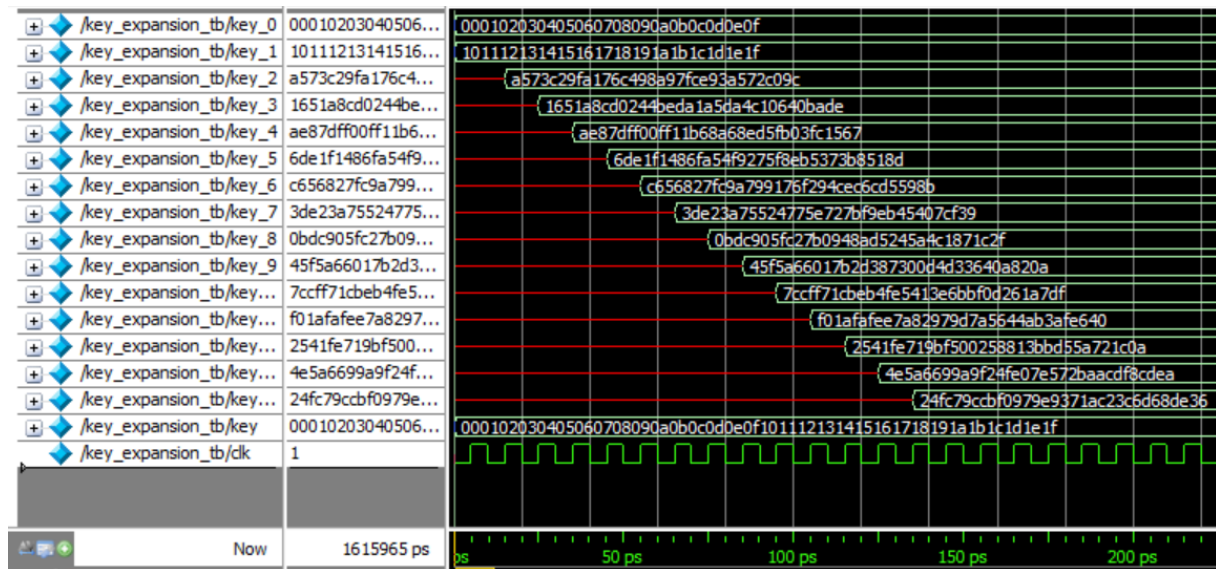
end
endmodule

```

1.5.2. Kết quả

Mô tả	Input	Key	Output (theo FIPS 197)	Output module đã thiết kế	Đánh giá
Round 0 ark	001122334455 66778899aabb ccddeeff	00010203040506 0708090a0b0c0d 0e0f	0010203040506070 8090a0b0c0d0e0f0	00102030405060 708090a0b0c0d0 e0f0	Đúng
Round 1 s-box	001020304050 60708090a0b0 c0d0e0f0	10111213141516 1718191a1b1c1d 1e1f	63cab7040953d051c d60e0e7ba70e18c	63cab7040953d0 51cd60e0e7ba70e 18c	Đúng
Round 1 s-row	63cab7040953 d051cd60e0e7 ba70e18c		6353e08c0960e104c d70b751bacad0e7	6353e08c0960e1 04cd70b751baca d0e7	Đúng
Round 1 m-col	6353e08c0960 e104cd70b751 bacad0e7		5f72641557f5bc92f 7be3b291db9f91a	5f72641557f5bc9 2f7be3b291db9f9 1a	Đúng
Round 1 ark	5f72641557f5 bc92f7be3b29 1db9f91a		4f63760643e0aa85e fa7213201a4e705	4f63760643e0aa8 5efa7213201a4e7 05	Đúng
Round 2 s-box	4f63760643e0 aa85efa72132 01a4e705	a573c29fa176c49 8a97fce93a572c0 9c	84fb386f1ae1ac97df 5cfd237c49946b	84fb386f1ae1ac9 7df5cfd237c4994 6b	Đúng
Round 2 s-row	84fb386f1ae1a c97df5cfd237c 49946b		84e1fd6b1a5c946fdf 4938977cfbac23	84e1fd6b1a5c946 fdf4938977cfbac 23	Đúng
Round 2 m-col	84e1fd6b1a5c 946fdf493897 7cfbac23		bd2a395d2b6ac438d 192443e615da195	bd2a395d2b6ac4 38d192443e615d a195	Đúng
Round 2 ark	bd2a395d2b6a c438d192443e 615da195		1859fbc28a1c00a07 8ed8aad42f6109	1859fbc28a1c00a 078ed8aad42f61 09	Đúng
Round 3 s-box	1859fbc28a1c 00a078ed8aad c42f6109	1651a8cd0244bed a1a5da4c10640ba de	adcb0f257e9c63e0b c557e951c15ef01	adcb0f257e9c63e 0bc557e951c15ef 01	Đúng
Round 3 s-row	adcb0f257e9c 63e0bc557e95 1c15ef01		ad9c7e017e55ef25b c150fe01ccb6395	ad9c7e017e55ef2 5bc150fe01ccb63 95	Đúng
Round 3 m-col	ad9c7e017e55 ef25bc150fe01 ccb6395		810dce0cc9db8172b 3678c1e88a1b5bd	810dce0cc9db81 72b3678c1e88a1 b5bd	Đúng
Round 3 ark	810dce0cc9db 8172b3678c1e 88a1b5bd		975c66c1cb9f3fa8a9 3a28df8ee10f63	975c66c1cb9f3fa 8a93a28df8ee10f 63	Đúng
Round 4 s-box	975c66c1cb9f 3fa8a93a28df8 ee10f63	ae87dff00ff11b68 a68ed5fb03fc156 7	884a33781fdb75c2d 380349e19f876fb	884a33781fdb75 c2d380349e19f8 76fb	Đúng
Round 4 s-row	884a33781fdb 75c2d380349e 19f876fb		88db34fb1f807678d 3f833c2194a759e	88db34fb1f80767 8d3f833c2194a7 59e	Đúng
Round	88db34fb1f80		b2822d81abe6fb275	b2822d81abe6fb	Đúng

4 m-col	7678d3f833c2194a759e		faf103a078c0033	275faf103a078c0033	
Round 4 ark	b2822d81abe6fb275faf103a078c0033		1c05f271a417e04ff921c5c104701554	1c05f271a417e04ff921c5c104701554	Đúng



Hình 19: Kết quả mô phỏng ModelSim của testbench KeyExpansion.v

```
# Initial Key: 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
# Key 0: 000102030405060708090a0b0c0d0e0f
# Key 1: 101112131415161718191a1b1c1d1e1f
# Key 2: a573c29fa176c498a97fce93a572c09c
# Key 3: 1651a8cd0244bedala5da4c10640bade
# Key 4: ae87dff00ff11b68a68ed5fb03fc1567
# Key 5: 6de1f1486fa54f9275f8eb5373b8518d
# Key 6: c656827fc9a799176f294cec6cd5598b
# Key 7: 3de23a75524775e727bf9eb45407cf39
# Key 8: 0bdc905fc27b0948ad5245a4c1871c2f
# Key 9: 45f5a66017b2d387300d4d33640a820a
# Key 10: 7ccff71cbeb4fe5413e6bbf0d261a7df
# Key 11: f01afafee7a82979d7a5644ab3afe640
# Key 12: 2541fe719bf500258813bbd55a721c0a
# Key 13: 4e5a6699a9f24fe07e572baacdf8cdea
# Key 14: 24fc79ccbf0979e9371ac23c6d68de36
```

Hình 20: Kết quả display output ModelSim của testbench MixColumns.v

1.6. So sánh kết quả testbench module AddRoundKey, SubBytes, ShiftRows, MixColumns với một số test vector của tài liệu FIPS 197

1.7. So sánh kết quả testbench module Key Expansion với test vector của tài liệu FIPS 197

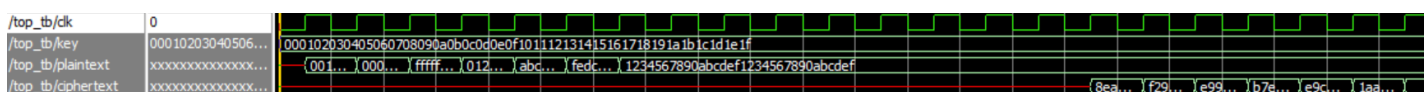
Key ban đầu:

000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f

Round	Output Key theo tài liệu FIPS 197	Output Key của module đã thiết kế	Đánh giá
0	000102030405060708090a0b0c0d0e0f	000102030405060708090a0b0c0d0e0f	Đúng
1	101112131415161718191a1b1c1d1e1f	101112131415161718191a1b1c1d1e1f	Đúng
2	a573c29fa176c498a97fce93a572c09c	a573c29fa176c498a97fce93a572c09c	Đúng
3	1651a8cd0244beda1a5da4c10640bade	1651a8cd0244beda1a5da4c10640bade	Đúng
4	ae87dff00ff11b68a68ed5fb03fc1567	ae87dff00ff11b68a68ed5fb03fc1567	Đúng
5	6de1f1486fa54f9275f8eb5373b8518d	6de1f1486fa54f9275f8eb5373b8518d	Đúng
6	c656827fc9a799176f294cec6cd5598b	c656827fc9a799176f294cec6cd5598b	Đúng
7	3de23a75524775e727bf9eb45407cf39	3de23a75524775e727bf9eb45407cf39	Đúng
8	0bdc905fc27b0948ad5245a4c1871c2f	0bdc905fc27b0948ad5245a4c1871c2f	Đúng
9	45f5a66017b2d387300d4d33640a820a	45f5a66017b2d387300d4d33640a820a	Đúng
10	7ccff71cbeb4fe5413e6bbf0d261a7df	7ccff71cbeb4fe5413e6bbf0d261a7df	Đúng
11	f01afafee7a82979d7a5644ab3afe640	f01afafee7a82979d7a5644ab3afe640	Đúng
12	2541fe719bf500258813bbd55a721c0a	2541fe719bf500258813bbd55a721c0a	Đúng
13	4e5a6699a9f24fe07e572baacdf8cdea	4e5a6699a9f24fe07e572baacdf8cdea	Đúng
14	24fc79ccbf0979e9371ac23c6d68de36	24fc79ccbf0979e9371ac23c6d68de36	Đúng

1.8. Kết quả mô phỏng kiến trúc Pipeline

Trong file mô phỏng kiến trúc pipeline, tại mỗi cạnh lên của xung clock sẽ có một plaintext mới được đưa vào khối top. Có tổng cộng 7 test case được đưa vào và kết quả của mỗi test case đó được xuất ra sau 15 chu kỳ xung clock.



Hình 21: Kết quả mô phỏng ModelSim của testbench top.v

```

Time 5: Input PlainText[0] = 00112233445566778899aabbccddeeff
-----
Time 15: Input PlainText[1] = 00000000000000000000000000000000
-----
Time 25: Input PlainText[2] = ffffffffffffffffffffffffffffffff
-----
Time 35: Input PlainText[3] = 0123456789abcdef0123456789abcdef
-----
Time 45: Input PlainText[4] = abcdef0123456789abcdef0123456789
-----
Time 55: Input PlainText[5] = fedcba9876543210fedcba9876543210
-----
Time 65: Input PlainText[6] = 1234567890abcdef1234567890abcdef
-----
Time 165: Output CipherText[0] = 8ea2b7ca516745bfeafc49904b496089
-----
Time 175: Output CipherText[1] = f29000b62a499fd0a9f39a6add2e7780
-----
Time 185: Output CipherText[2] = e999e41d4ca770da5387117b5d8f57ee
-----
Time 195: Output CipherText[3] = b7e7f4e4da5004021090a21cdf555652
-----
Time 205: Output CipherText[4] = e9c21842535fe2075b4adde47721ec8b
-----
Time 215: Output CipherText[5] = 1aa5c5dce72f7127e0571abb38543282
-----
Time 225: Output CipherText[6] = bff4952d10c8e25f1cd5e7e7d53f954f
-----

```

Hình 22: Kết quả display output ModelSim của testbench top.v

C.3 AES-256 ($Nk=8, Nr=14$)

```

PLAINTEXT: 00112233445566778899aabbccddeeff
KEY:       000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
round[14].output 8ea2b7ca516745bfeafc49904b496089

```

Hình 23: Kết quả tính toán lý thuyết của thuật toán (nist.fips.197)

II. Đánh giá tổng quát

2.1. Pre-simulation.

Trong giai đoạn kiểm thử mức Logic (Pre-simulation), project không sử dụng Testbench Verilog truyền thống mà áp dụng sự kết hợp giữa verilog và python. Các công nghệ cốt lõi bao gồm:

- **Ngôn ngữ Python:** Điều khiển toàn bộ luồng kiểm thử, sinh dữ liệu ngẫu nhiên và đối soát kết quả. Python giúp tự động hóa quá trình thay vì phải viết các vector test bằng Verilog.
- **Framework Cocotb:** Đây là công cụ giúp kết nối mã Python với phần mềm mô phỏng phần cứng (Simulator). Cocotb cung cấp các hàm API (như Clock, RisingEdge) để tương tác trực tiếp với các chân tín hiệu (ports) của module Verilog theo thời gian thực mô phỏng.
- **Thư viện PyCryptodome:** Được sử dụng như một *Golden Reference Model*. Kết quả mã hóa từ thư viện này dùng để so sánh với kết quả do mạch Verilog tính toán ra.

- **Phần mềm mô phỏng lỗi (Simulator):** Cocotb ở lớp trên sẽ gọi ngầm Icarus Verilog, công cụ mô phỏng phần cứng bên dưới để biên dịch và chạy file .v.

```
got = await _run_one(dut, NIST_KEY, NIST_PLAINTEXT)
record(got == NIST_CIPHERTEXT, 0, NIST_CIPHERTEXT, got, NIST_KEY,
NIST_PLAINTEXT, "nist")

random_tests = int(os.getenv("AES_RANDOM_TESTS", "0"))
seed = int(os.getenv("AES_SEED", "1"))

if random_tests > 0:
    aes = _get_pycryptodome_cipher()
    if aes is None:
        dut._log.warning("pycryptodome not installed, skipping random
tests")
    else:
        rng = random.Random(seed)
        for i in range(random_tests):
            key = _rand_bytes(rng, 32)
            plaintext = _rand_bytes(rng, 16)
            expected = aes.new(key, aes.MODE_ECB).encrypt(plaintext)
            got = await _run_one(dut, key, plaintext)
            record(got == expected, i + 1, expected, got, key, plaintext,
"random")
```

Bài test đầu tiên luôn sử dụng bộ Khóa (Key) và Bản rõ (Plaintext) chuẩn xác định bởi NIST để kiểm tra logic sơ bộ.

Các bài test tiếp theo (test thứ 2 đến 1005) được sinh hoàn toàn ngẫu nhiên bằng thư viện random của Python. Các byte ngẫu nhiên này được đồng thời đưa vào hàm AES của PyCryptodome để lấy trước đáp án dự kiến (*expected*).

Sau 15 xung clock, kết quả tại ngõ ra dut.ciphertext.value được đọc về và so sánh trực tiếp với đáp án chuẩn (*expected*).

```
out_path = Path(os.getenv("AES_RESULTS_JSON", "results/sim_cases.json"))
if not out_path.is_absolute():
    out_path = Path(__file__).resolve().parents[1] / out_path
out_path.parent.mkdir(parents=True, exist_ok=True)

payload = dict(results)
payload["failures"] = failures
out_path.write_text(json.dumps(payload, indent=2))

csv_path = Path(os.getenv("AES_CASES_CSV", "results/sim_cases.csv"))
if not csv_path.is_absolute():
    csv_path = Path(__file__).resolve().parents[1] / csv_path
csv_path.parent.mkdir(parents=True, exist_ok=True)
with csv_path.open("w", newline="") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=["index", "type", "key", "plaintext", "expected", "got", "pass"],
    )
    writer.writeheader()
    writer.writerows(cases)
```


Kết quả Pass/Fail được hệ thống tự động ghi nhận vào file báo cáo (*sim_cases.json* và *sim_cases.csv*).

Câu lệnh Pre-simulation:

```
python scripts/run_sim_cocotb.py --sim icarus --tests 1005 --seed 7 --results-dir results/presimulation
```

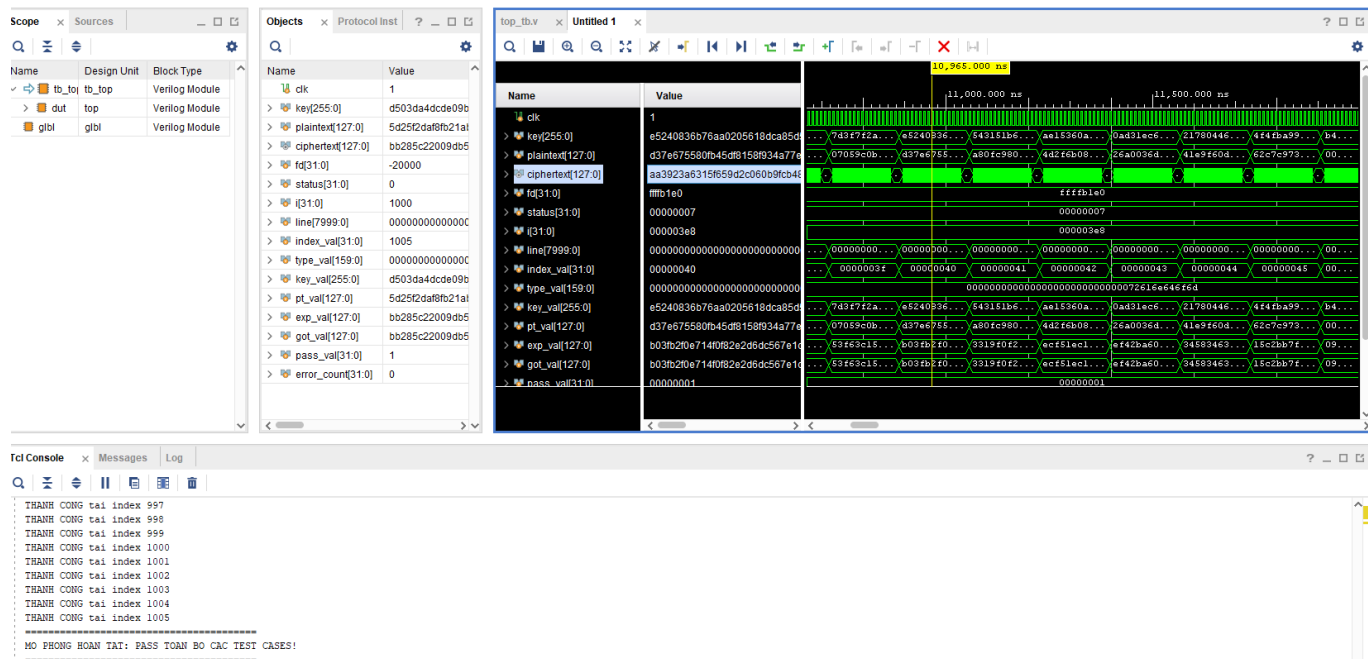
File Explorer view showing the directory structure: This PC > OD (D:) > STUDY > HE_THONG_HDL > DO_AN > code > AES256_Copy > results > presimulation.

Name	Date modified	Type	Size
sim_cases.csv	5/19/2026 9:17 PM	Microsoft Excel C...	175 KB
sim_cases.json	5/19/2026 9:17 PM	JSON Source File	1 KB

Hình 24: Kết quả pass/fail sẽ được lưu vào file *sim_cases.csv* và *sim_cases.json*

```
1 {
2   "cases_total": 1006,
3   "cases_passed": 1006,
4   "cases_failed": 0,
5   "failures": []
6 }
```

Hình 25: Kết quả kiểm tra file *sim_cases.json*: 100% case đã được mã hóa chính xác



Hình 27: Kết quả mô phỏng trên vivado

2.2 Post-simulation.

Quy trình Post-simulation của dự án được tự động hóa qua các script Python, bao gồm 4 bước chính:

1. **Tổng hợp & Thực thi (Synthesis & Implementation):** Tạo ra Netlist và file độ trễ SDF.
2. **Mô phỏng Gate-Level với SDF:** Chạy mô phỏng trên Netlist với độ trễ thực tế để kiểm tra chức năng và tạo file VCD.
3. **Phân tích từ VCD:** Dùng file VCD để tính toán các thông số chính xác.
4. **Tổng hợp và Vẽ biểu đồ:** Trích xuất dữ liệu từ các báo cáo và vẽ biểu đồ tổng kết.

Bước 1: Tổng hợp & Thực thi (Synthesis & Implementation)

- **Mục đích:** Chuyển đổi mã Verilog (RTL) thành một bản thiết kế ở mức cổng logic (gate-level netlist) mà công cụ có thể triển khai trên FPGA. Quá trình này cũng tạo ra file **SDF** (*Standard Delay Format*) chứa thông tin độ trễ của các cổng và đường dây.
- **File liên quan:**
 - Script điều khiển: `scripts/run_vivado.py`
 - Script thực thi (Tcl): `scripts/run_synth_impl.tcl` (được gọi bởi script Python)
- **Input:**
 - Toàn bộ các file mã nguồn Verilog (`top.v`, `encrypt_round.v`, `KeyExpansion.v`, v.v.).
- **Output:**

+ Nhóm file Báo cáo (Reports):

- `results/timing_summary.rpt`: Báo cáo chi tiết về định thời mạch (Timing summary) như WNS, TNS.
- `results/utilization.rpt`: Báo cáo việc sử dụng tài nguyên trên FPGA (số lượng

LUTs, Registers, v.v.).

- results/power.rpt: Báo cáo ước tính công suất tiêu thụ của mạch.
- + **Nhóm file Mô phỏng mức Cổng (Gate-level / Post-Implementation Simulation):**
- results/aes256_timesim.v: File Verilog Netlist mức cổng (gate-level) có bao gồm thiết lập delay thông qua SDF annotation.
 - results/aes256_timesim_noanno.v: File Verilog Netlist nhưng không bao gồm SDF annotation.
 - results/aes256_timesim.sdf: File Standard Delay Format chứa thông tin về độ trễ (delay) thực tế của các cell và wire sau khi đã nối dây (routing) thành công.

```
python scripts/run_vivado.py
```

```

33 1. CLB Logic
34 -----
35
36 +-----+-----+-----+-----+-----+-----+
37 | Site Type | Used | Fixed | Prohibited | Available | Util% |
38 +-----+-----+-----+-----+-----+-----+
39 | CLB LUTs* | 13198 | 0 | 0 | 4085760 | 0.32 |
40 | LUT as Logic | 13198 | 0 | 0 | 4085760 | 0.32 |
41 | LUT as Memory | 0 | 0 | 0 | 956160 | 0.00 |
42 | CLB Registers | 1920 | 0 | 0 | 8171520 | 0.02 |
43 | Register as Flip Flop | 1920 | 0 | 0 | 8171520 | 0.02 |
44 | Register as Latch | 0 | 0 | 0 | 8171520 | 0.00 |
45 | CARRY8 | 0 | 0 | 0 | 510720 | 0.00 |
46 | F7 Muxes | 4384 | 0 | 0 | 2042880 | 0.21 |
47 | F8 Muxes | 2113 | 0 | 0 | 1021440 | 0.21 |
48 | F9 Muxes | 0 | 0 | 0 | 510720 | 0.00 |
49 +-----+-----+-----+-----+-----+-----+

```

Hình 28: Báo cáo Sử dụng Tài nguyên

LUTs (Look-Up Tables): Sử dụng 13,198 LUTs (0.32%). Phần lớn số lượng LUT này được dùng để hiện thực hóa các bảng thay thế S-box (Substitute Bytes) và các phép toán logic trên trường Galois (MixColumns).

Registers (Flip-Flops): Sử dụng 1,920 Flip-Flops (0.02%): mạch sử dụng **15 thanh ghi 128-bit**, - đang được thiết kế dạng đường ống (Pipelined) với 14 vòng (rounds) mã hóa của AES-256 + 1 thanh ghi đếm.

```

4. I/O
-----
+-----+-----+-----+-----+-----+-----+
| Site Type | Used | Fixed | Prohibited | Available | Util% |
+-----+-----+-----+-----+-----+-----+
| Bonded IOB | 513 | 0 | 0 | 2072 | 24.76 |
+-----+-----+-----+-----+-----+-----+

```

Hình 29: Báo cáo sử dụng chân IO

I/O Ports (Pin): Sử dụng 513 chân IO (24.76%).

Module top nhận đầu vào song song. Đối với AES-256, một block dữ liệu gồm Data In (128-bit), Data Out (128-bit), Key (256-bit). Tổng cộng: 128 + 128 + 256 = 512 bit. Thêm 1 chân tín hiệu clk.

1. Summary

Total On-Chip Power (W)	9.858
Design Power Budget (W)	Unspecified*
Power Budget Margin (W)	NA
Dynamic (W)	3.922
Device Static (W)	5.936
Effective TJA (C/W)	0.4
Max Ambient (C)	95.7
Junction Temperature (C)	29.3
Confidence Level	Low
Setting File	---
Simulation Activity File	---
Design Nets Matched	NA

Total On-Chip Power (9.858 W): Tổng công suất tiêu thụ của toàn bộ chip. Bằng tổng của Dynamic Power và Device Static Power.

Dynamic Power (3.922 W): Công suất động. Đây là phần công suất sinh ra do quá trình chuyển mạch của các cổng logic, thanh ghi và sự dao động của xung nhịp (clock).

Device Static Power (5.936 W): Công suất tĩnh (hay còn gọi là *leakage power* - công suất rò rỉ). Đây là công suất bị tiêu hao ngay cả khi mạch không có xung nhịp, chủ yếu phụ thuộc vào công nghệ chế tạo silicon và nhiệt độ. Dòng chip UltraScale+ (xcvu19p) nhóm đang dùng rất lớn nên công suất rò rỉ khá cao.

Hình 26: Báo cáo công suất

Design Timing Summary

WNS(ns)	TNS(ns)	TNS Failing Endpoints	TNS Total Endpoints	WHS(ns)	THS(ns)
0.056	0.000	0	1792	0.129	0.000

Hình 30: Báo cáo định thời

WNS (Worst Negative Slack) (0.056 ns): Thời gian trễ dư (slack) nhỏ nhất trong tất cả các đường dẫn đối với Setup Time.

(*Setup Time là khoảng thời gian dữ liệu phải ổn định trước khi có sườn lên của clock.)

TNS (Total Negative Slack) (0.000 ns): Tổng của tất cả các Setup Slack bị vi phạm (âm) bằng 0, nghĩa là không có bất kỳ đường nào bị lỗi Setup.

WHS (Worst Hold Slack) (0.129 ns): Thời gian trễ dư nhỏ nhất đối với Hold Time. Hold Time là khoảng thời gian dữ liệu phải được giữ ổn định sau sườn lên của clock. WHS dương nghĩa là dữ liệu không bị trôi đi quá nhanh (không bị lỗi Hold).

THS (Total Hold Slack) (0.000 ns): Tương tự TNS nhưng dành cho Hold Time. Mạch không có vi phạm nào.

Max Delay Paths

Slack (MET) :	0.056ns (required time - arrival time)
Source:	state_reg[9][105]/C (rising edge-triggered cell FDRE clocked by clk {rise@0.000ns fall@0.975ns})
Destination:	state_reg[10][41]/D (rising edge-triggered cell FDRE clocked by clk {rise@0.000ns fall@0.975ns})
Path Group:	clk
Path Type:	Setup (Max at Slow Process Corner)
Requirement:	1.950ns (clk rise@1.950ns - clk rise@0.000ns)
Data Path Delay:	1.737ns (logic 0.576ns (33.162%) route 1.161ns (66.838%))
Logic Levels:	6 (LUT2=1 LUT5=1 LUT6=2 MUXF7=1 MUXF8=1)
Clock Path Skew:	-0.148ns (DCD - SCD + CPR)
Destination Clock Delay (DCD):	3.786ns = (5.736 - 1.950)

Hình 31: Đường dẫn dữ liệu dài nhất (Critical Path) trong mạch

Bước 2: Mô phỏng Gate-Level với độ trễ SDF (Post-Simulation)

- **Mục đích:** Đây là bước mô phỏng sau khi thực thi. Nó không chạy trên mã Verilog gốc nữa mà chạy trên Netlist (kết quả của Bước 1). Testbench sẽ đọc các ca kiểm thử (test case) từ file .csv (đã được tạo ở bước pre-simulation), áp dụng chúng vào Netlist có kèm độ trễ từ file SDF, và ghi lại toàn bộ hoạt động của các tín hiệu ra file *VCD* (*Value Change Dump*).
- **File liên quan:**
 - Script điều khiển: *scripts/run_gate_sim.py*
 - Testbench (SystemVerilog): *tests/gate_sdf_tb.sv*
- **Input:**
 - *results/postSimulation/aes256_timesim_noanno.v* (từ Bước 1)
 - *results/postSimulation/aes256_timesim.sdf* (từ Bước 1)
 - *results/presimulation/sim_cases.csv* (File chứa các test case từ pre-simulation)
- **Output:**
 - **File hoạt động sóng:** *results/postSimulation/aes256.vcd*
 - Log mô phỏng .csv hiển thị PASS hoặc FAIL cho từng test case.

```
python scripts/run_gate_sim.py
```

index	type	key	plaintext	expected	got	pass
0	nist	000102030	001122334	8ea2b7ca	8ea2b7ca	1
1	random	52f22665a	a095f20f9	7e3b8edd	7e3b8edd	1
2	random	248a1e924	745c4c3fc	4f479c74e	4f479c74e	1
3	random	ba72499b	94cc7411c	3a67e2e23	3a67e2e23	1
4	random	a593feaed	142a7266f	e5f0a6f81	e5f0a6f81	1
5	random	5baee261f	83f39ea7a	06ea75ee	06ea75ee	1
6	random	646566641	9d6026a2c	369eb1e8e	369eb1e8e	1
7	random	777a7b4f1	dd17b2d8	538d3b25	538d3b25	1
8	random	54a2399cc	f45872cee	de21871e3	de21871e3	1
9	random	3256347b5	5516cdf2f	6b1746e2e	6b1746e2e	1
10	random	20072697e	6ffcdf31d3	b443d238c	b443d238c	1
11	random	428b6bd5	01c6cc262	20c557d10	20c557d10	1
12	random	8e7bc8c61	738288ce7	a079af20a	a079af20a	1
13	random	8fe4f133d	e223f7773	821d8c131	821d8c131	1
14	random	29b46efe8	c93af8e01	23cc86255	23cc86255	1
15	random	6cd9e9ad1	159bdb38	f895cda56	f895cda56	1
16	random	449f210b8	40090304f	43a35a86	43a35a86	1
17	random	d1a66ea8	12a0bde1	c00387b59	c00387b59	1
18	random	48993eb14	794780a73	72b9cb09e	72b9cb09e	1
19	random	0a64054c4	0bd3d5b7	a21f83d8b	a21f83d8b	1
20	random	8191d5d0c	a088ae3e7	8b59c407c	8b59c407c	1
21	random	a88610be1	a43213992	bca39126f	bca39126f	1
22	random	0f7c44f8a	d181f7fe7	e11af4c65	e11af4c65	1
23	random	24bf8643f	246a58605	600173ed4	600173ed4	1
24	random	ed32b603e	3ff8446f82	9c5975dde	9c5975dde	1
25	random	a166e9e0f	4c411bdbc	db418ca0	db418ca0	1
26	random	2aa429133	9133e305f	39539c1a3	39539c1a3	1
27	random	0f7f4793f	7086cb5c3e	d2e66cbd	d2e66cbd	1
28	random	d387da77	3b91eb09	25dad12a	25dad12a	1
29	random	1c19124c8	f569b4a64	13848b479	13848b479	1
30	random	aa6cec5e3	3843c2e34	ed1aaae0	ed1aaae0	1
31	random	aa0ef2982	2fa7ef86b	3db671a1	3db671a1	1
32	random	2b1b0014	31525dbcc	1503a766c	1503a766c	1
33	random	087610cd	eaceae2f2f	808d36b6	808d36b6	1
34	random	62ca40a9f	3264c0283	805fa6877	805fa6877	1

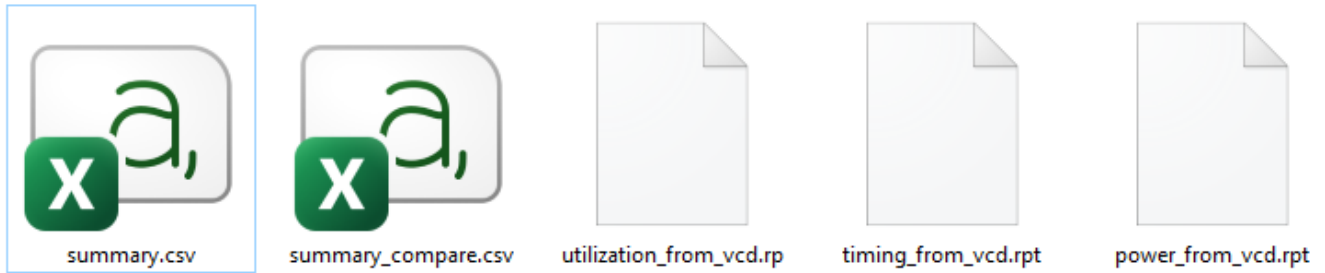
Hình 32: File CSV kết quả (Post-Simulation CSV)

Bước 3: Phân tích data từ VCD.

- **Mục đích:** Báo cáo mặc định của Vivado chỉ là ước tính. Bằng cách sử dụng file VCD từ Bước 2, ta cung cấp cho Vivado thông tin chính xác (VD: về tần suất chuyển mạch (switching activity) của từng tín hiệu trong điều kiện chạy thực tế). Điều này giúp tạo ra một báo cáo với độ chính xác cao hơn nhiều.
- **File liên quan:**
 - Script điều khiển: *scripts/run_power_from_vcd.py*
 - Script thực thi (Tcl): *scripts/run_power_from_vcd.tcl* (được gọi bởi script Python)
- **Input:**
 - *results/postSimulation/aes256.vcd* (từ Bước 2)
 - Project Vivado đã thực thi xong: *project_1/project_1.xpr*
- **Output:**
 - + **Báo cáo công suất chính xác:**
 - *power_from_vcd.rpt*: Báo cáo tiêu thụ năng lượng (dựa trên hoạt động chuyển đổi tín hiệu thực tế lấy từ file VCD).
 - *timing_from_vcd.rpt*: Báo cáo timing (Setup/Hold time, WNS, TNS...).

- *utilization_from_vcd.rpt*: Báo cáo tài nguyên sử dụng phần cứng của FPGA (LUTs, Flip-Flops,...).

python scripts/run power_from_vcd.py

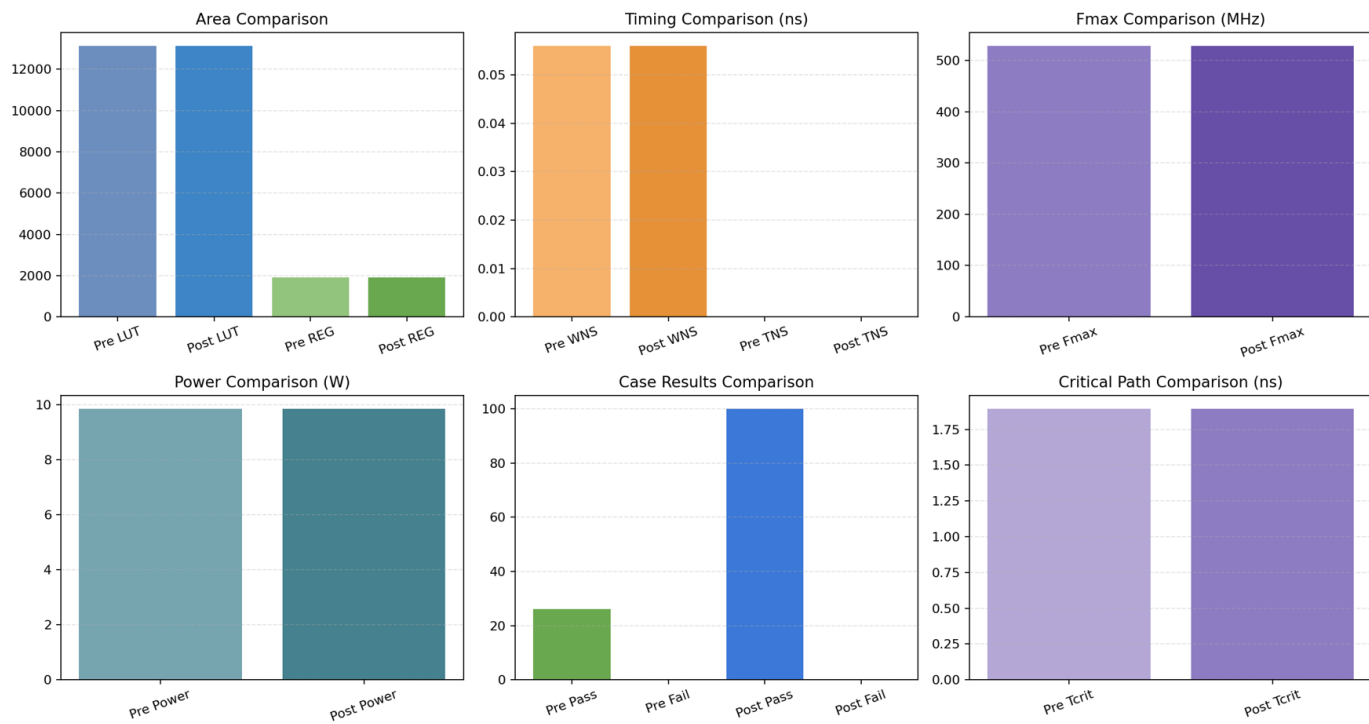


Hình 33: Các file sau khi được tạo

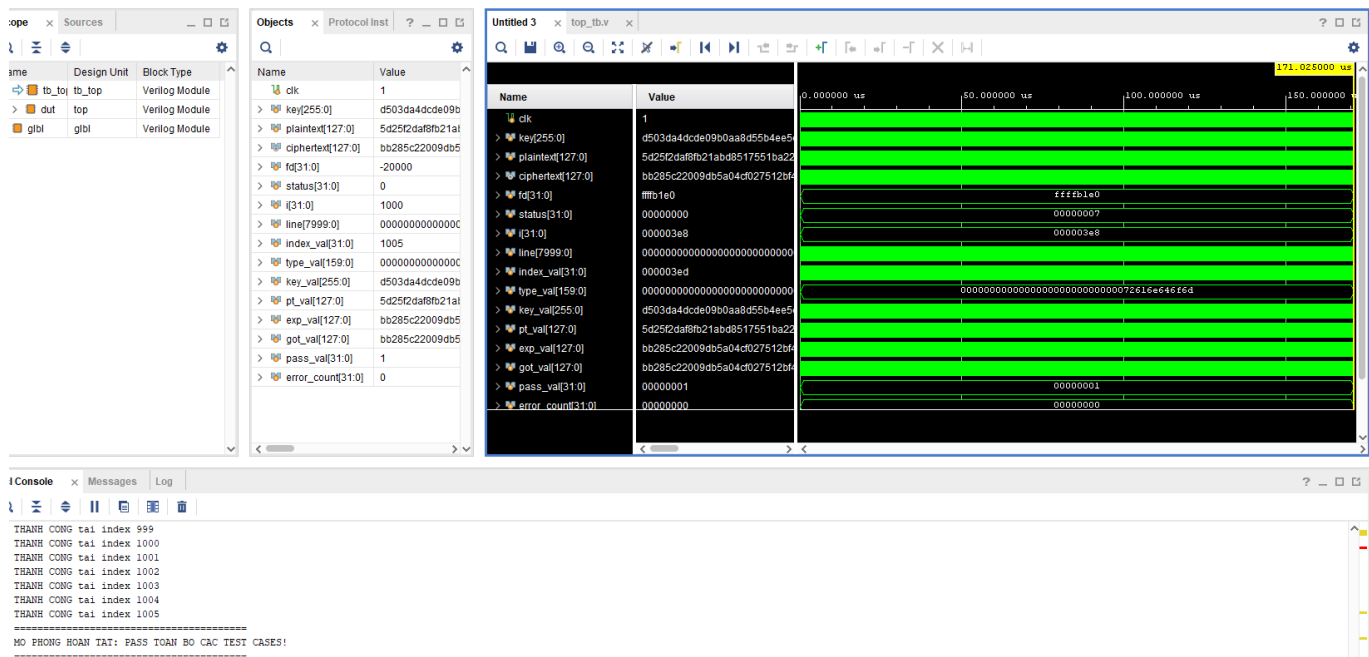
Bước 4: Tổng hợp và Vẽ biểu đồ báo cáo

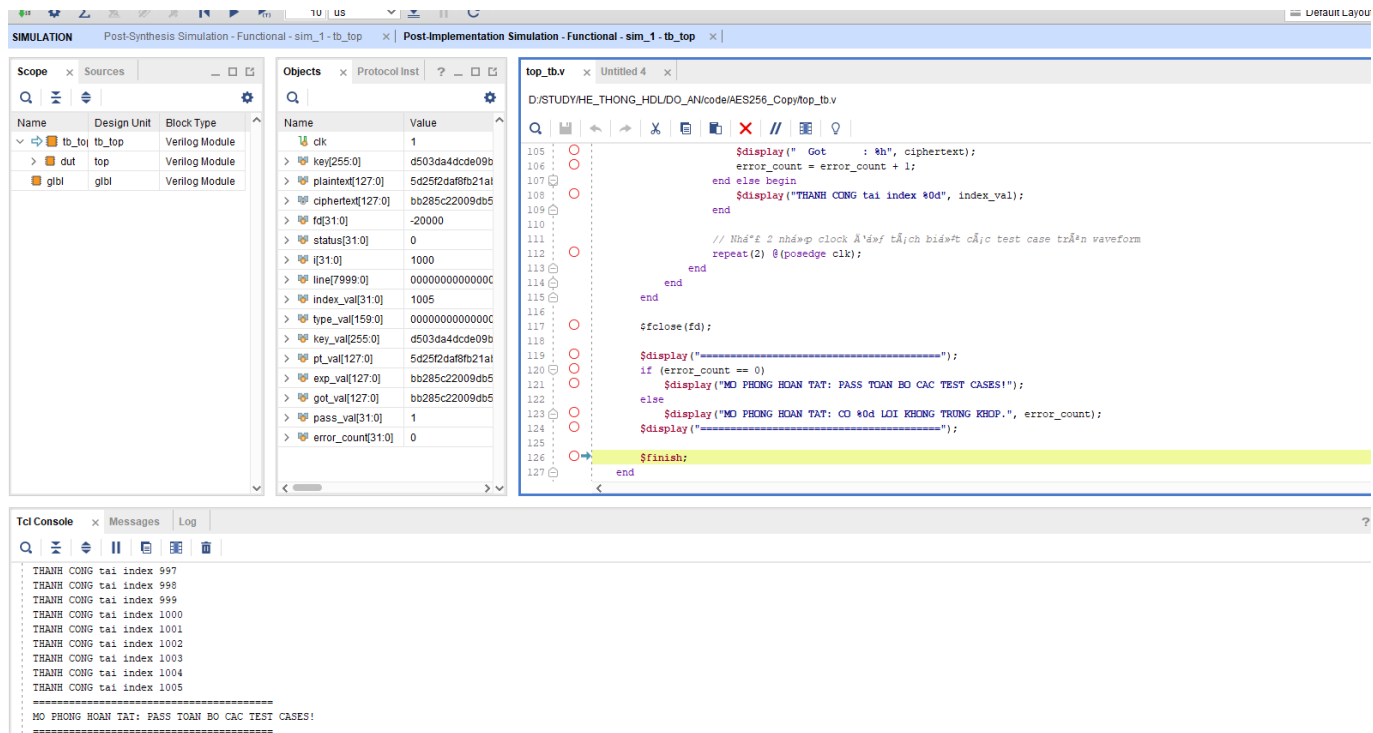
- **Mục đích:** Sau khi có tất cả các file báo cáo (.rpt) và kết quả mô phỏng (.json), các script này sẽ tự động đọc, trích xuất các con số quan trọng (tài nguyên, timing, công suất, pass/fail) vào một file summary.csv, và cuối cùng là vẽ biểu đồ tổng kết.
- **File liên quan:**
 - scripts/parse_reports.py
 - scripts/plot_reports.py
- **Input:**
 - Các file: timing_summary.rpt, utilization.rpt, power_from_vcd.rpt.
 - File kết quả mô phỏng: results/sim_cases.json.
- **Output:**
 - File dữ liệu tổng hợp: results/summary.csv
 - **Biểu đồ kết quả cuối cùng:** results/summary.png

python scripts/parse_reports.py
python scripts/plot_reports.py

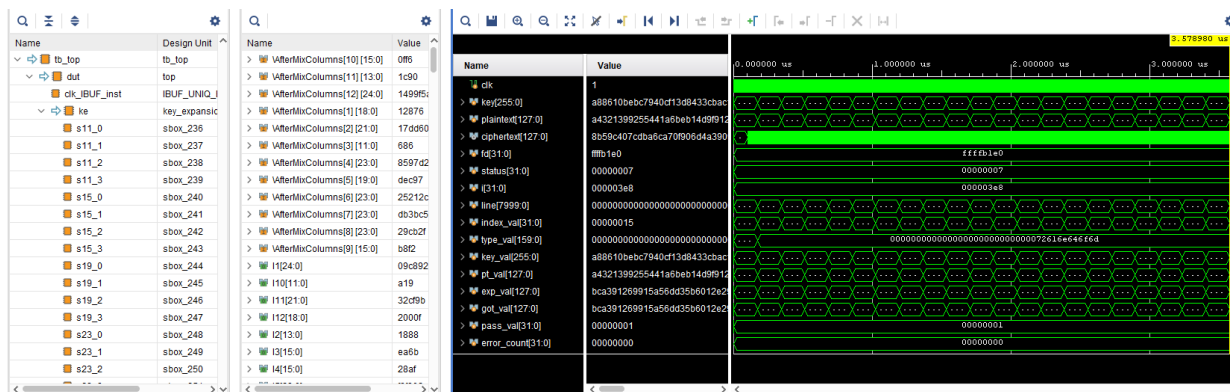


Hình 34: Các biểu đồ so sánh chi tiết giữa Pre-Simulation và Post-Simulation từ file `summary_compare.csv`.





Hình 35_a: Run simulation synthesis/implementation post functional



Hình 35_b: Run timing post simulation implementation

III. Total coverage code.

Line Coverage Score 50
Branch Coverage Score 50
Condition Coverage Score 0
Toggle Coverage Score 0.73

Code Coverage Report Generation done

Hình 36: Tổng coverage code là 73 %

CHƯƠNG 4: TỔNG KẾT VÀ ĐÁNH GIÁ QUÁ TRÌNH THỰC HIỆN

I. Kết quả đạt được của đề tài

Quá trình nghiên cứu và triển khai đề tài đã hoàn thành xuất sắc mục tiêu cốt lõi là thiết kế thành công một lõi mã hóa AES-256 hoàn toàn tuân thủ các đặc tả kỹ thuật khắt khe của tiêu chuẩn FIPS 197. Toàn bộ thiết kế đã được mô tả ở mức thanh ghi chuyển mức RTL thông qua ngôn ngữ phân cứng Verilog.

Bằng việc ứng dụng kiến trúc Fully Unrolled Pipeline cho toàn bộ 14 vòng lặp mã hóa, hệ thống đã giải quyết triệt để bài toán thất cổ chai về mặt hiệu năng thường gặp trong các thiết kế lặp vòng Iterative truyền thống. Tính chính xác của thiết kế đã được minh chứng một cách thuyết phục thông qua quá trình Unit Testing và mô phỏng Testbench. Bằng việc sử dụng bộ Test Vector chuẩn do tổ chức NIST cung cấp, mọi tín hiệu ngõ ra từ module AddRoundKey cho đến bản mã Ciphertext cuối cùng đều khớp hoàn toàn với dữ liệu lý thuyết trên phần mềm mô phỏng.

II. Đánh giá hiệu năng và tài nguyên hệ thống

Dựa trên nguyên lý hoạt động của kiến trúc phân tầng, thiết kế phân cứng cho thấy một sự đánh đổi Trade-off rõ rệt giữa tài nguyên logic và tốc độ luồng dữ liệu.

Về mặt tiêu thụ tài nguyên, việc trải phẳng toàn bộ cấu trúc mã hóa đòi hỏi việc khởi tạo đồng thời 14 khối SubBytes và 13 khối MixColumns trên đường ống dữ liệu. Điều này dẫn đến sự gia tăng đáng kể số lượng phần tử logic và diện tích cổng logic trên vi mạch FPGA.

Tuy nhiên, sự đánh đổi này mang lại một hiệu năng xử lý luồng dữ liệu vượt trội. Khối Key Expansion được thiết kế hoạt động song song và cấp phát khóa liên tục cho từng tầng mã hóa. Nhờ vậy, sau khoảng thời gian trễ Latency ban đầu là 15 chu kỳ xung nhịp Clock, hệ thống đạt được thông lượng Throughput tối đa bằng cách liên tục xuất ra một khối dữ liệu Ciphertext 128-bit mới tại mỗi chu kỳ kế tiếp. Đặc tính này biến thiết kế trở thành một giải pháp lý tưởng cho các ứng dụng đòi hỏi băng thông mã hóa thời gian thực.

III. Những hạn chế còn tồn tại

Dù đạt được những kết quả khả quan về mặt tối ưu tốc độ và sự chính xác logic, quá trình thực hiện đề tài vẫn còn một số điểm giới hạn nhất định. Đề tài mới chỉ dừng lại ở bước kiểm chứng Functional Simulation thông qua giản đồ xung Waveform và quá trình tổng hợp Synthesis trên phần mềm. Thiết kế chưa được nạp trực tiếp xuống một board mạch FPGA thực tế để đo đạc các thông số vật lý như công suất tiêu thụ điện năng tĩnh và động.

IV. Định hướng phát triển tương lai

Từ những hạn chế đã được phân tích, nghiên cứu này mở ra nhiều hướng phát triển tiềm năng cho giai đoạn tối ưu hóa tiếp theo nhằm đưa thiết kế tiến gần hơn tới các tiêu chuẩn công nghiệp:

- **Đóng gói IP Core:** Trọng tâm đầu tiên sẽ là việc xây dựng các bộ điều khiển

Controller và tích hợp giao tiếp chuẩn công nghiệp để đóng gói lõi AES-256 thành một module IP Core hoàn chỉnh có khả năng cắm và chạy Plug-and-Play trên các vi điều khiển nhúng.

- **Tăng cường an ninh phần cứng:** Thiết kế sẽ được nâng cấp bằng các kỹ thuật che giấu Masking hoặc chèn nhiễu đồng hồ phân cứng Clock Jitter để tăng cường khả năng kháng lại các đợt tấn công dò xét năng lượng.

CHƯƠNG 5: TÀI LIỆU THAM KHẢO

[1] *M.S.Abdul-Karim , K.H.Rahouma , and K.Nasr, “Effective Pipelined FPGA Implementation for AES-256 Algorithm”.*

[2] *Withdrawn NIST Technical Series Publication.*

[3] *C.Sreevalli , K.Dharvathu , Dr.M.S.Anuradha , SRF CSIR –NET 2 , A.Professor, “FPGA IMPLEMENTATION OF AESALGORITHM USING VERILOG-HDL”.*

[4] *<https://www.w3schools.com/python/>*